

Detecting and Localizing Segment-Level Poisoning in Multi-Source LLM-Agent Inputs

Xue Tan*, Changhui Wang*, Sanrui Yang*, Hao Luan*,
Zhuyang Yu*, Jin Wei*, Ping Chen*, Xiaoyan Sun[†], and Jun Dai[†]
*Fudan University
[†]Worcester Polytechnic Institute

Abstract

Modern Large Language Model (LLM) agents increasingly construct prompts by aggregating external data segments, such as retrieved passages, user reviews, and documents from independent sources. While this multi-source paradigm enables flexible information fusion, it also exposes LLMs to segment-level poisoning attacks, where an adversary controlling only a small subset of sources can inject malicious segments to steer model outputs toward attacker-desired goals. Existing defenses mainly rely on surface textual patterns, external embeddings, or auxiliary detectors, making them vulnerable to fluent and semantically plausible poisoned segments. Moreover, most defenses focus on binary detection and provide limited support for identifying the responsible segments. We shift the defensive signal from external text to the LLM’s internal representation space. Our key observation is that successful poisoning attacks, including corrupted-evidence and adversarial-instruction attacks, induce structured activation shifts associated with malicious generation. Under calibration, we empirically observe a consistent activation-space pattern, which we term the *poison direction*. Based on this insight, we propose ACTPROBE, an internal-state-based framework for detecting and localizing poisoned segments in multi-source LLM inputs. ACTPROBE projects high-dimensional MLP activations onto the learned poison direction and uses a lightweight linear SVM to detect contaminated prompts from a small calibration set. Once contamination is detected, ACTPROBE invokes BINROL, which combines recursive replacement ablation, Mahalanobis-distance-based branch pruning, and MAD-based robust leaf detection to efficiently identify poisoned segments. This design does not modify the backend LLM and reduces localization overhead from exhaustive $O(n)$ probing to $O(k \cdot \log n)$ forward passes. Across three datasets, two attacks, and four open-weight LLMs, ACTPROBE achieves low detection errors (0.01 FPR and 0.05 FNR) and strong localization performance (0.94 Recall and 0.90 F1-score). It also remains effective under defense-aware adaptive attacks and can protect black-box APIs through surrogate-based poisoned-segment removal.

1 Introduction

Large language models (LLMs) are increasingly used as the reasoning core of autonomous agents that aggregate information from multiple external sources to complete user tasks. In a typical multi-source setting, an agent constructs its prompt by concatenating a task instruction with n data segments, each supplied by an independent external source. This paradigm is common in modern LLM applications. Question-answering agents rely on retrieval-augmented generation (RAG) [18, 32] to collect passages from external knowledge bases as supporting evidence, where each passage may originate from a different document whose correctness cannot be directly verified by the agent. Content summarization agents face a similar setting: e-commerce review agents [27] aggregate reviews from independent users to generate purchase recommendations, while news briefing agents [26] synthesize articles from different outlets into a unified summary. Here, the LLM processes external segments whose reliability and intent are not fully controlled by the service provider.

This dependence on external segments creates a fundamental security risk. An adversary who controls even a small fraction of the input sources can inject poisoned segments that substantially steer the model’s generation. As a result, the agent may present fabricated or attacker-controlled content as if it were supported by trustworthy evidence. For example, a poisoned passage may cause a question-answering agent to incorrectly state that Mount Fuji is the highest mountain in the world, or manipulate an e-commerce summarizer into producing unwarranted warnings about severe product side effects, potentially causing reputational and financial harm. Recent studies have demonstrated the feasibility of such segment-level poisoning from two complementary perspectives: *knowledge poisoning* and *prompt injection*. Knowledge poisoning attacks [16, 41, 43] inject factually incorrect yet semantically plausible segments into external knowledge sources, causing the model to rely on corrupted evidence during generation. For instance, PoisonedRAG [43] shows that optimized poisoned passages can be retrieved for target queries and induce

attacker-specified responses. In contrast, prompt injection attacks [11, 19, 33] embed adversarial instructions within external data segments to override the agent’s intended task. ObliInjection [33] shows that malicious segments can remain effective even when their positions among benign segments are uncertain. Although these paradigms differ in whether they manipulate factual evidence or task instructions, both exploit the same weakness: after prompt integration, a few malicious external segments can corrupt the final LLM output.

Existing defenses provide only partial protection against segment-level poisoning. Prevention-based methods [6, 7] harden LLMs through fine-tuning or safety alignment, but require modifying the target model, may reduce benign-task utility [14], and remain vulnerable to adaptive attacks; for example, ObliInjection retains attack success rates of 53%–78% against hardened models [33]. More importantly, a practical defense for multi-source applications must solve two coupled problems: determining whether an input is contaminated and identifying which external segments are responsible. Detection alone cannot support targeted sanitization, while localization is unreliable without an accurate contamination signal. Existing methods struggle with both objectives because they primarily rely on textual appearance or model-external representations. PPL [2, 12] and KAD [19] exhibit false negative rates of up to 92.6% against ObliInjection [33]; EVD relies on general-purpose embeddings disconnected from the target LLM’s computation; and DataSentinel [20], despite training a dedicated detector, still misses 79.6% of ObliInjection-contaminated inputs. Localization methods are similarly limited: PromptLocate [13] uses a text-level oracle to identify malicious instructions, making it suitable for explicit prompt injection but less effective against data-only poisoning containing plausible misinformation. Consequently, existing defenses struggle to reliably detect and localize fluent, semantically coherent poisoned segments.

To address this limitation, we shift the defensive signal from external text to the LLM’s internal representation space. We observe that successful segment-level poisoning attacks, whether based on corrupted evidence or adversarial instructions, induce activation shifts that consistently align along a geometric direction across samples, which we call the *poison direction*. Figure 1 illustrates this effect: after projection onto the learned direction, poisoned inputs exhibit a clear shift from clean inputs in both the all-layer average and the most discriminative layer. This structured activation-space footprint offers a more direct signal of adversarial influence than surface-level text features. However, turning this insight into a practical defense introduces two challenges. First, modern LLM activations are extremely high-dimensional; for example, the MLP activations of Llama-3.1-8B-Instruct contain 14,336 dimensions per layer. Directly training detectors on such representations is computationally costly and prone to overfitting when only limited calibration data are available. Second, localizing k poisoned segments among n inputs can

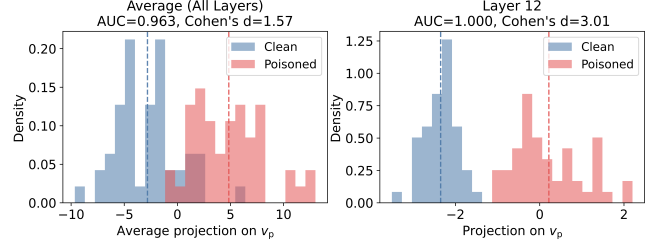


Figure 1: Projection distributions of clean and poisoned activations along the poison direction. Results are shown for Mistral-7B-Instruct-v0.3 on Amazon Reviews under ObliInjection.

require many forward passes under segment-wise testing, and dense poisoning can cause malicious signals to mask one another, making attribution difficult.

We propose ACTPROBE, an internal-state-based framework for detecting and localizing poisoned segments in multi-source LLM inputs. ACTPROBE addresses the two challenges above through two coordinated phases. To address the high-dimensional detection challenge, the **Detection Phase** learns a poison direction from a small calibration set and projects high-dimensional MLP activations onto this direction, producing a compact layer-wise feature representation. A lightweight linear SVM then classifies whether the assembled prompt is contaminated, while the same calibration process estimates the benign activation distribution for later reuse. To address the efficient localization challenge, once a prompt is flagged as suspicious, the **Localization Phase** invokes BINROL (Binary Search with Robust Leaf Detection). BINROL recursively partitions the input context and applies replacement ablation, substituting candidate segments with neutral text and measuring the resulting reduction in Mahalanobis distance to the benign distribution. This process efficiently narrows down suspicious regions without exhaustive segment-wise testing. At the leaf level, BINROL applies Median Absolute Deviation (MAD)-based robust outlier detection to identify the final poisoned segments, partially reducing the impact of mutual masking in dense poisoning settings. ACTPROBE avoids modifying the backend LLM, relies on internal activation rather than surface text heuristics, and reduces localization overhead from exhaustive $O(n)$ probing to $O(k \cdot \log n)$ forward passes.

We evaluate ACTPROBE on HotpotQA, MS-MARCO, and Amazon Reviews across four open-weight LLMs and two multi-source attacks, PoisonedRAG and ObliInjection. ACTPROBE consistently outperforms text-level, embedding-based, and localization baselines.

In summary, our main contributions are as follows:

- We propose ACTPROBE, an internal-state-based defense that learns a *poison direction* from calibration data in the LLM activation space to detect contaminated multi-source inputs and localize the responsible segments.

- We design BINRoL, which combines recursive replacement ablation, Mahalanobis-based branch pruning, and MAD-based robust leaf detection to localize poisoned segments with $O(k \cdot \log n)$ forward passes.
- Across three datasets, two attacks, and four open-weight LLMs, ACTPROBE outperforms existing baselines with average detection FPR/FNR of 0.01/0.05 and localization Recall/F1-score of 0.94/0.90. It also demonstrates robustness against adaptive attacks and characterizes partial transferability across attack settings.

2 Related Work

2.1 Large Language Models (LLMs)

Large Language Models (LLMs) are autoregressive generative models typically based on the Transformer architecture. Given an input sequence X , an LLM models the conditional probability of an output sequence $Y = \{y_1, y_2, \dots, y_m\}$ in a token-by-token manner:

$$P(Y|X) = \prod_{t=1}^m P(y_t|X, y_{<t}). \quad (1)$$

In multi-source applications, the input X is constructed by concatenating segments from multiple external sources. Let x_i denote the i -th segment; then the input can be written as $X = [x_1; x_2; \dots; x_n]$. Through self-attention, the model integrates information across these segments to generate a response aligned with the task objective. Table 11 in the Appendix A summarizes the key notations used in this paper.

2.2 Attacks on Multi-Source LLM Applications

In multi-source LLM applications, adversaries often control only a small fraction of external segments, yet these segments can still influence the final output. Existing attacks mainly follow two paradigms: knowledge poisoning and prompt injection. Knowledge poisoning compromises external knowledge sources by injecting factually incorrect but semantically coherent content, causing the model to rely on corrupted evidence during generation [28, 40, 41]. PoisonedRAG [43] is a representative RAG poisoning attack that inserts optimized passages into the knowledge base to increase retrieval relevance to target queries and induce attacker-specified responses. Other attacks, such as PR-Attack [16] and BadRAG [36], further improve poisoning effectiveness by optimizing triggers, poisoned texts, or retriever-generator interactions. Prompt injection instead embeds adversarial instructions into external segments to override the intended task or instruction hierarchy [11, 19, 34]. In multi-source settings, malicious segments may appear at arbitrary positions

among benign ones, making position-dependent attacks unreliable. ObliInjection [33] addresses this challenge with an order-oblivious objective and the order GCG algorithm, producing malicious segments that remain effective under different segment permutations. Other attacks, such as Neural Exec [24] and JudgeDeceiver [29], extend instruction hijacking to retrieval-augmented and evaluation-oriented scenarios. In summary, knowledge poisoning corrupts factual evidence, whereas prompt injection manipulates task instructions. We adopt PoisonedRAG and ObliInjection as representative attacks to evaluate detection and localization under evidence-level and instruction-level poisoning.

2.3 Defenses against Segment-Level Poisoning Attacks

Existing defenses against segment-level poisoning in LLM applications mainly fall into three categories: prevention, detection, and localization. Prevention-based methods, such as StruQ [6] and SecAlign [7], harden LLMs through instruction tuning or safety alignment to separate trusted instructions from untrusted data. However, they require modifying the target model, may affect benign-task utility [14], and primarily target instruction-level attacks rather than knowledge poisoning. Detection-based methods identify contaminated inputs or segments without changing the backend LLM. Perplexity-based filtering [2, 12] uses language-model likelihood, Known-Answer Detection [19, 23] probes instruction-following behavior with predefined checks, embedding-based detectors flag semantic outliers in external representation spaces, and DataSentinel [20] trains a dedicated detector through adversarial optimization. Despite their differences, these methods rely on surface statistics, external embeddings, or auxiliary models rather than the protected LLM’s internal computation. Localization-based defenses further aim to identify the malicious segments after contamination is detected. Prompt-Locate [13] localizes injected prompts through semantic segmentation and oracle-style detection, but is tailored to explicit prompt-injection patterns. General attribution methods, such as feature-removal attribution [39] and Shapley-value attribution [21], can be adapted for localization, but either perturb the input structure or incur high computational cost. In contrast, ACTPROBE leverages internal activations to provide a unified defense for detecting and localizing malicious segments under both knowledge poisoning and prompt injection.

3 Problem Formulation

3.1 Multi-Source Information Fusion in LLM Agents

In modern LLM agent applications, particularly in *Information Fusion* tasks, the input context is frequently aggregated from multiple independent sources. We refer to the discrete

data contributed by each source as a *segment*. Formally, consider n sources, where source i provides a text segment x_i . The target context x^t is formed by concatenating these n segments in a specific order determined by the service provider’s backend logic:

$$x^t = x_{i_1} \| x_{i_2} \| \cdots \| x_{i_n}, \quad (2)$$

where $\{i_1, i_2, \dots, i_n\}$ is a permutation of $\{1, 2, \dots, n\}$. The LLM f receives a prompt $p = s^t \| x^t$, where s^t is a system instruction specifying the intended task, and generates a response $r^t = f(s^t \| x^t)$.

This formulation captures knowledge-aggregation applications, including Retrieval-Augmented Generation (RAG) [18, 43] and multi-document summarization (e.g., e-commerce reviews) [27]. A key property of these tasks is the *majority-consensus assumption*: the LLM implicitly functions as a consensus builder, synthesizing information across segments, where benign content is expected to dominate and establish a stable contextual baseline. From a security perspective, the service provider aggregates segments from sources that are not trusted. This design exposes the LLM’s consensus mechanism to dense *Sybil attacks*, where a single adversary injects multiple malicious segments that masquerade as independent sources and exert disproportionate influence over the model’s output. Even without knowledge of the segment permutation, an adaptive adversary can exploit this vulnerability by introducing a small but strategically crafted subset of adversarial segments to bias the final output.

3.2 Threat Model

Attacker’s Goal. We consider an adversary who aims to manipulate the output of a multi-source LLM application by injecting malicious content into a subset of external segments. Let $x^t = x_{i_1} \| x_{i_2} \| \cdots \| x_{i_n}$ denote the clean context assembled from n external segments, and let s^t denote the task instruction. The benign response is $r^t = f(s^t \| x^t)$. The attacker contaminates a subset $A \subseteq \{1, \dots, n\}$ of segments, where $|A| = k$ and $k \geq 1$, producing a contaminated context x^c by replacing each clean segment x_i for $i \in A$ with an adversarial segment $\tilde{x}_i$. The attacker’s goal is to induce the LLM to generate an attacker-specified response r^e instead of the intended benign response r^t . We consider the attack successful if

$$f(s^t \| x^c) \simeq r^e, \quad (3)$$

where $\simeq$ denotes semantic equivalence. For example, the generated answer may endorse a targeted false fact, or the generated summary may promote a fabricated sentiment.

Attacker’s Capabilities and Knowledge. The adversary can forge, upload, or compromise k out of the n external sources, such as malicious reviews, poisoned web pages, or corrupted documents. We assume $k < n/2$, reflecting practical compromise costs and preserving benign-source dominance [35]. We further evaluate larger values of k in Section 5.3.2 and provide a detailed analysis.

The adversary may have different levels of model access depending on the attack method. For open-weight models, poisoned segments may be crafted with white-box access; for closed-source services, the adversary may rely on black-box queries or surrogate models. However, the adversary does not control the service provider’s runtime prompt assembly process. In particular, the adversary does not know the exact benign segments contributed by legitimate sources, nor the final segment permutation $\{i_1, i_2, \dots, i_n\}$ used to construct the context. The adversary also cannot tamper with the defender’s calibration data or observe runtime internal activations. Therefore, effective attacks must remain robust to unknown benign context, uncertain segment ordering, and private activation-space parameters.

3.3 Defender Model

Defense Goal. The defender aims to prevent poisoned external segments from manipulating the LLM’s final response. To this end, the defense performs two tasks before the final response is generated:

- *Detection*: Given an assembled prompt $p = s^t \| x$, determine whether the context contains any adversarial segment. This is a prompt-level binary classification task.
- *Localization*: If the prompt is detected as contaminated, identify the specific segment indices responsible for the attack. Formally, the defender outputs a suspicious set $\hat{D} \subseteq \{1, \dots, n\}$. The localized segments can then be removed or replaced before the final response is generated.

Defense Capabilities and Assumptions. We assume the defender operates under the following conditions:

1. *Segment Boundary Access*: The defender has access to the segment boundaries used by the multi-source application. This is natural for RAG, review aggregation, and multi-document summarization systems, where the service provider explicitly concatenates discrete external segments into the prompt.
2. *Diagnostic Activation Access*: For open-weight LLMs, the defender can extract hidden-layer activations from the protected model. For closed-source API models, the defender can instead deploy an open-weight LLM as an upstream diagnostic surrogate to inspect and sanitize the context before forwarding it to the black-box model. We evaluate this surrogate-based deployment in Section 6.1.
3. *Calibration Data and Coverage*: The defender has access to a small calibration set containing M clean prompts and M poisoned prompts, with $M = 100$ in our experiments. Poisoned samples are generated using known attacks or red-teaming and are assumed to cover representative attack families and data domains expected at deployment.

time, rather than exact runtime attack instances. ACTPROBE’s performance depends on this coverage and may degrade against unseen attacks with substantially different activation patterns. As attacks evolve, the defender can update the calibration set with new attack samples and recalibrate the detector.

4 Our ACTPROBE

In this section, we present ACTPROBE, a two-stage framework engineered to detect and localize poisoned segments within multi-source LLM inputs. Following a high-level architectural overview (Section 4.1), we define the poison direction probe that underpins the entire framework (Section 4.2). We subsequently detail the operational mechanics of the Poison Prompt Detection stage (Section 4.3) and the Poison Segment Localization stage (Section 4.4).

4.1 Overview

The workflow of ACTPROBE consists of offline calibration and online inference, as illustrated in Figure 2 and formalized in Algorithm 1 in the Appendix A.

Offline Calibration. Leveraging a modest calibration dataset of M clean and M poisoned prompts, ACTPROBE establishes its defensive boundaries by learning three core components: (1) a *poison direction probe* $\mathbf{v}_p$ that projects high-dimensional LLM activations onto a subspace highly sensitive to adversarial manipulation (Section 4.2); (2) a linear SVM classifier tailored for prompt-level detection (Section 4.3); and (3) clean distribution parameters (μ, Σ^{-1}) derived as a byproduct of the profiling process. Crucially, these parameters are directly reused during the localization stage without incurring any additional computational overhead.

Online Inference. Upon receiving a newly assembled prompt $p = s^t \| x_1 \| \cdots \| x_n$, ACTPROBE initiates the *Poison Prompt Detection* stage. The prompt’s internal activations are extracted, projected via $\mathbf{v}_p$, and evaluated by the SVM. If classified as benign, the inspection terminates, and the LLM permits standard generation. Conversely, if an anomaly is flagged, ACTPROBE triggers the *Poison Segment Localization* stage. Here, the BINROL algorithm (Algorithm 2) systematically isolates all contaminated segments utilizing a recursive binary search coupled with robust leaf-node anomaly detection, efficiently pinpointing the poisoned sources with an expected cost of only $O(k \cdot \log n)$ forward passes.

4.2 Poison Direction Probe

The hidden representations of modern LLMs reside in exceptionally high-dimensional spaces (e.g., the MLP activation dimension is $d = 14,336$ per layer across $L = 32$ transformer blocks in Llama-3.1-8B-Instruct, resulting in a raw feature

space of $d \times L \approx 460,000$ dimensions). Training anomaly detectors directly on these raw activations is highly problematic for two primary reasons. First, it requires learning millions of parameters, thereby necessitating prohibitively large datasets to avert severe overfitting. This data requirement is fundamentally incompatible with real-world defensive scenarios, where rapidly acquiring extensive labeled samples is practically infeasible. Second, raw activations are entangled with substantial task-relevant semantic noise, which can easily obscure the subtle anomalies induced by malicious inputs. To overcome these statistical and representational bottlenecks, we introduce the *poison direction probe*. This component is designed as a targeted dimensionality reduction mechanism, specifically engineered to isolate the dominant activation shift exclusively induced by poisoning.

Specifically, given a calibration set of M paired clean and poisoned prompts, we extract the last-token hidden activation $\mathbf{h}_{i,l} \in \mathbb{R}^d$ at each layer l for each prompt i . We use the last-token activation because it serves as a compact prompt-level representation for decoder-only LLMs immediately before next-token prediction, and has been widely adopted in activation probing and representation engineering [5, 31, 42]. The poison direction at layer l is calculated as the normalized mean difference between the adversarial and benign representations:

$$\mathbf{v}_l = \frac{1}{M} \sum_{i=1}^M (\mathbf{h}_{i,l}^p - \mathbf{h}_{i,l}^c), \quad \mathbf{v}_p[l] = \frac{\mathbf{v}_l}{\|\mathbf{v}_l\|}, \quad (4)$$

where $\mathbf{h}_{i,l}^p$ and $\mathbf{h}_{i,l}^c$ denote the activations of the i -th poisoned and clean prompt at layer l , respectively. By projecting any given activation $\mathbf{h}_l$ onto this unit vector, we obtain a scalar projection $s_l = \mathbf{h}_l \cdot \mathbf{v}_p[l]$. Consequently, the feature representation for an entire prompt is constructed as:

$$\mathbf{s} = [s_1, s_2, \dots, s_L] \in \mathbb{R}^L. \quad (5)$$

Through this projection, the representation at each layer is distilled from d dimensions down to a single scalar, effectively compressing the full network activation into a compact L -dimensional feature vector. Unlike generic dimensionality reduction techniques (e.g., PCA [1]) that predominantly capture directions of maximum variance, $\mathbf{v}_p$ isolates a mechanistically grounded subspace that explicitly captures the perturbation footprint of adversarial poisoning. This targeted projection preserves the critical adversarial signal while systematically filtering out task-irrelevant semantic noise. Consequently, a higher projection magnitude s_l directly quantifies the severity of the malicious activation shift, endowing the final feature vector $\mathbf{s}$ with both exceptional discriminative power and inherent interpretability. This discriminative capability is visually confirmed in Appendix B (Figures 8 and 9). We empirically verify this advantage in Section 5.5.2, demonstrating that $\mathbf{v}_p$ substantially outperforms random projections and PCA in detection and localization.

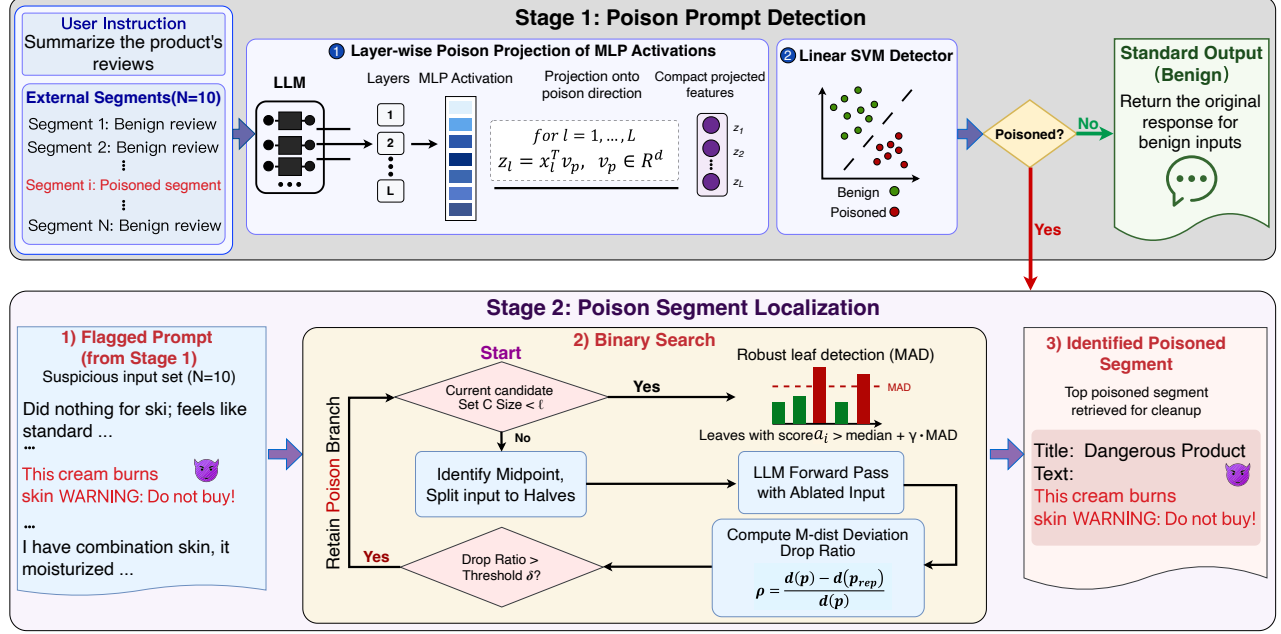


Figure 2: Overview of ACTPROBE. The detection phase identifies poisoned prompts via projection-based SVM classification, and the subsequent localization phase isolates malicious segments with BINROL.

4.3 Poison Prompt Detection

The Poison Prompt Detection stage determines whether an assembled prompt is contaminated. We formulate this as a binary classification task operating on the compact L -dimensional projection features derived in Section 4.2.

Classifier Design. The poison direction probe distills the hidden states into a compact feature vector (e.g., $L=32$ dimensions for Llama-3.1-8B-Instruct) that is explicitly aligned with the adversarial perturbation. Within this low-dimensional and well-structured representation space, a linear decision boundary is empirically sufficient and computationally efficient. We employ a linear Support Vector Machine (SVM) [8] for its maximum-margin formulation, which provides favorable generalization properties in low-dimensional feature spaces even with limited training data (e.g., $M=100$ benign and $M=100$ poisoned calibration samples for $L=32$ features). We empirically substantiate this design choice by benchmarking the linear SVM against alternative classification algorithms in Section 5.5. Concretely, after standardizing the projection features to zero mean and unit variance via a fitted scaler, the classification rule is defined as:

$$\hat{y} = \text{sign}(\mathbf{w}^\top \tilde{\mathbf{s}} + b), \quad (6)$$

where $\tilde{\mathbf{s}} \in \mathbb{R}^L$ is the standardized feature vector, $\mathbf{w} \in \mathbb{R}^L$ is the learned weight vector, and b is the bias term. Since the features are standardized, the magnitude $|w_l|$ provides a comparable estimate of the contribution of layer l to poisoning detection. We use these layer-wise weights to select the most

discriminative layers for the localization stage.

Baseline Distribution Profiling. Fitting the SVM concurrently yields a critical statistical byproduct required for the subsequent localization stage: the distributional parameters of benign activations. Using the unstandardized projection features of the M clean calibration prompts, we estimate the empirical mean and covariance:

$$\mu = \frac{1}{M} \sum_{i=1}^M \mathbf{s}_i^c, \quad \Sigma = \text{Cov}(\{\mathbf{s}_i^c\}_{i=1}^M) + \lambda \mathbf{I}, \quad (7)$$

where $\lambda=10^{-6}$ serves as a standard Tikhonov regularization term to guarantee the invertibility of Σ . The parameters (μ, Σ^{-1}) define the normative boundary of benign prompts within the projection space. They are stored for direct cross-phase reuse during localization, acting as the reference distribution for Mahalanobis distance computations without incurring any additional calibration overhead. The validity of this parameter reuse is grounded in the structural robustness of the poison direction $\mathbf{v}_p$ against neutral text replacements.

Online Inference Pipeline. During active deployment, processing a new prompt requires only a single forward pass through the LLM. The extracted MLP activations are projected onto $\mathbf{v}_p$ to generate $\mathbf{s}$, standardized to $\tilde{\mathbf{s}}$, and evaluated by the SVM. If the prompt is classified as benign, the defensive inspection terminates immediately, imposing negligible computational overhead on the standard generation process. Conversely, if an adversarial perturbation is detected, the system automatically invokes the Poison Segment Localization stage to pinpoint the malicious segments.

4.4 Poison Segment Localization via BINROL

Upon a positive flag from the detection stage, the localization phase determines *which* of the n segments are poisoned. We propose BINROL (**B**inary search with **R**obust **L**ean detection), which decomposes localization into two phases: a coarse binary narrowing phase that efficiently reduces the candidate set, followed by a fine-grained leaf detection phase that precisely isolates the poisoned segments. The procedure is formalized in Algorithm 2 in the Appendix B.

4.4.1 Replacement Ablation

A straightforward localization strategy is to directly inspect each segment’s activation projection and flag segments with unusually large scores. However, this strategy is unreliable in autoregressive LLMs, where the hidden state at each position is conditioned on all preceding tokens. As a result, the poisoning signal introduced by a contaminated segment can *propagate forward* through the sequence, causing subsequent benign segments to inherit traces of the malicious perturbation. This signal leakage makes it difficult to distinguish the segment that originates the poisoning effect from those that merely transmit its influence.

To address this issue, we adopt *replacement ablation*. Specifically, we replace a target segment or segment group with neutral text (e.g., “No relevant information is available for this query.”; see Appendix A.1 for the complete list of templates) and perform a new forward pass. This operation preserves the segment slot and the surrounding prompt structure while suppressing the semantic and adversarial content of the replaced segment. By comparing the LLM’s internal states before and after replacement, we obtain a cleaner estimate of the replaced segment’s contribution to the poisoning signal, rather than relying on its raw activation projection, which may be confounded by forward propagation effects. We empirically validate the superiority of this replacement-ablation design over direct deletion-based ablation in Section 5.5.3.

4.4.2 Drop Ratio

We measure the effect of replacement ablation by quantifying how much the prompt representation moves toward the clean distribution after candidate segments are replaced. Let $\mathbf{s}_{\text{full}}$ and $\mathbf{s}_{\text{rep}}$ denote the projection features before and after replacement, respectively. We compute the Mahalanobis distance [22] to the clean distribution as

$$d(\mathbf{s}) = \sqrt{(\mathbf{s} - \mu)^\top \Sigma^{-1} (\mathbf{s} - \mu)}, \quad (8)$$

where (μ, Σ^{-1}) are the benign distribution parameters estimated during the detection stage (Eq. 7). The *drop ratio* is then defined as the relative reduction in anomaly score:

$$\rho = \frac{d(\mathbf{s}_{\text{full}}) - d(\mathbf{s}_{\text{rep}})}{d(\mathbf{s}_{\text{full}})}. \quad (9)$$

A larger ρ indicates that replacing the candidate segments substantially reduces the poisoning-induced deviation, suggesting that these segments contribute strongly to the malicious signal. Since ρ is normalized by the original anomaly score, it mitigates variations caused by different prompt contents and lengths, reducing the need for per-sample threshold calibration.

4.4.3 The BINROL Algorithm

BINROL (Algorithm 2) consists of two complementary procedures: a coarse-grained binary narrowing stage and a fine-grained leaf detection stage.

Phase 1: BinaryNarrow. This procedure recursively reduces the candidate set from n segments to at most ℓ segments, where ℓ is the leaf size. At each recursion level, the current candidate set C is split into two halves, $\mathcal{L}$ and $\mathcal{R}$. Each half is independently replaced with neutral text, and the corresponding drop ratios $\rho_{\mathcal{L}}$ and $\rho_{\mathcal{R}}$ are computed. The recursion then follows three branching rules:

- Both $\rho_{\mathcal{L}} > \delta$ and $\rho_{\mathcal{R}} > \delta$, both branches are explored, indicating that poisoned segments may exist in both halves.
- Only one half exceeds δ , only that branch is explored.
- If neither half exceeds δ , the branch with the larger ρ is selected as a fallback to avoid prematurely discarding potential poisoned segments.

The branching threshold δ is tuned on an independent validation set and set to a permissive value (e.g., $\delta = 0.02$) to prioritize candidate recall during coarse narrowing, leaving fine-grained discrimination to the robust leaf detection stage.

Phase 2: RobustLeaf. Once the candidate set satisfies $|C| \leq \ell$, BINROL switches from group-level narrowing to per-segment attribution. For each segment $i \in \mathcal{P}(C)$, the parent candidate set of the current leaf C , we perform individual replacement ablation and compute its *ablation score*:

$$a_i = \frac{1}{|\mathcal{L}_{\text{top}}|} \sum_{l \in \mathcal{L}_{\text{top}}} (\mathbf{h}_l^{\text{full}} - \mathbf{h}_{i,l}^{\text{rep}}) \cdot \mathbf{v}_p[l], \quad (10)$$

where $\mathbf{h}_l^{\text{full}}$ denotes the MLP activation at layer l for the original prompt, and $\mathbf{h}_{i,l}^{\text{rep}}$ denotes the activation after replacing segment i with neutral text. The set $\mathcal{L}_{\text{top}}$ contains the most discriminative layers, selected according to the magnitudes of the SVM weights learned during detection, with its size specified as a validation-tuned hyperparameter in Section 5.1. Although prompt-level detection uses all layers to capture global evidence, segment-level attribution requires a cleaner and more localized signal. Focusing on $\mathcal{L}_{\text{top}}$ suppresses weakly informative layers that may introduce semantic noise, while also reducing the cost of repeated ablation scoring. A larger a_i indicates that replacing segment i removes a stronger component of the poisoning-induced activation shift, suggesting that this segment contributes more to the signal.

To identify anomalous candidates within a leaf, we apply a robust outlier detection rule based on the Median Absolute Deviation (MAD). Rather than computing MAD only within the current leaf $\mathcal{C}$, we estimate the robust statistics from the parent candidate set $\mathcal{P}(\mathcal{C})$, i.e., the candidate set before the last binary split:

$$\tilde{m} = \text{median}(\{a_i\}_{i \in \mathcal{P}(\mathcal{C})}), \quad \hat{\sigma} = \eta \cdot \text{median}(\{|a_i - \tilde{m}|\}_{i \in \mathcal{P}(\mathcal{C})}), \quad (11)$$

where $\eta = 1.4826$ makes MAD a consistent estimator of the standard deviation under a Gaussian distribution [17]. A candidate segment $i \in \mathcal{C}$ is flagged as poisoned if

$$a_i > \tilde{m} + \gamma \cdot \hat{\sigma}, \quad (12)$$

where γ controls the sensitivity of the attribution rule. Parent-level MAD provides a more stable threshold than leaf-level MAD when multiple poisoned segments fall into the same small leaf, while remaining more localized than global thresholding over all segments. If no candidate exceeds the threshold, BINROL selects the segment with the largest ablation score as a fallback, preserving localization recall when attribution signals are weak or ambiguous.

Complexity. In BINARYNARROW, each active node requires two forward passes for replacement ablation on its left and right halves, and the recursion depth is at most $\lceil \log_2(n/\ell) \rceil$. In ROBUSTLEAF, ACTPROBE computes ablation scores over the parent candidate set $\mathcal{P}(\mathcal{C})$ and applies the MAD threshold to the current leaf $\mathcal{C}$. Since $\mathcal{P}(\mathcal{C})$ is the immediate parent of a leaf, its size is bounded by $O(\ell)$ under balanced splitting. Therefore, when k poisoned segments activate k search paths, the total localization cost is

$$O(k(\log(n/\ell) + \ell)), \quad (13)$$

which reduces to $O(k \log n)$ for a constant leaf size ℓ .

5 Evaluation

5.1 Experimental Setup

Datasets. We evaluate ACTPROBE on three datasets spanning diverse multi-source scenarios, with statistics summarized in Table 12 in the Appendix A. *HotpotQA* [38] is a Wikipedia-based multi-hop QA benchmark; we construct each query with 6 Wikipedia segments. *MS-MARCO* [3] is a large-scale machine reading comprehension and passage retrieval dataset; we use 10 retrieved passages per query to instantiate RAG-style QA. *Amazon Reviews* [27] represents review summarization, where 50 user reviews per product are aggregated as external segments. These datasets cover both knowledge-grounded tasks and general multi-source aggregation. The prompt templates are shown in Figures 4 and 5.

Attacks. We evaluate ACTPROBE against two representative segment-level poisoning attacks. PoisonedRAG [43] is

a knowledge-poisoning attack that injects plausible but factually incorrect passages into the knowledge base, causing retrieved evidence to steer the LLM toward attacker-specified answers. ObliInjection [33] is a prompt-injection attack that uses an order-oblivious objective to craft adversarial segments that remain effective under different segment permutations.

Target LLMs. We evaluate ACTPROBE on four open-weight LLMs with accessible internal MLP activations: *Llama-3.1-8B-Instruct* [10], *Mistral-7B-Instruct-v0.3* [15], *GPT-OSS-20B*, and *Qwen3-4B-Instruct-2507* [37]. For closed-source API models, including *GPT-5.4-mini* and *Gemini-3.1-flash-Lite*, we use an open-weight LLM as an upstream diagnostic surrogate, as discussed in Section 6.1.

Baseline Defenses. We compare ACTPROBE with representative detection and localization baselines using the same input segments and, where applicable, calibration data. For detection, *PPL* [2, 12] flags segments whose language-model perplexity exceeds a predefined threshold; *KAD* [19, 23] prepends a known-answer detection prompt to each segment and identifies instruction interference when the expected answer is absent; *EVD* [13] encodes segments with a pre-trained sentence encoder and detects outliers in the resulting embedding space; and *DataSentinel* [20] uses a dedicated detection LLM trained through adversarial game-theoretic optimization, for which we evaluate the officially released model. For localization, *SFA* [25] projects each segment’s boundary-token activation onto the learned poison direction $\mathbf{v}_p$ and selects the highest-scoring segment; *FRA* [39] removes each segment individually and identifies the one causing the largest change in the model’s internal features or output behavior; *SVA* [21] uses Shapley values to estimate each segment’s marginal contribution to the poisoning signal; and *Prompt-Locate* [13] combines oracle-based contamination detection, contextual inconsistency analysis, and binary group search to locate the injected region.

Calibration and Hyperparameter Tuning. For each dataset-attack setting, we randomly sample 100 base instances without replacement and construct paired benign and poisoned prompts, yielding 100 prompts of each class. Calibration, validation, and evaluation splits are created before prompt construction and are disjoint in query/document identifiers for HotpotQA and MS-MARCO and in product/review identifiers for Amazon Reviews. This calibration set is used to learn the poison direction $\mathbf{v}_p$, train the linear SVM detector, and estimate the benign distribution parameters (μ, Σ^{-1}) . The remaining data are reserved for evaluation. To avoid data leakage, we tune all hyperparameters on an independent validation split: $\delta = 0.02$, $\gamma = 0.8$, $|\mathcal{L}_{\text{top}}| = 6$, and dataset-specific leaf sizes $\ell = 2, 3, 10$ for HotpotQA, MS-MARCO, and Amazon Reviews, respectively. By default, we use a poisoning ratio of 10%, i.e., $k = \lceil 0.1n \rceil$ poisoned segments per input, where n is the total number of segments. All reported results are averaged over five runs, with experiments and language model queries conducted on NVIDIA A6000 GPUs.

Dataset	Method	Llama-3.1-8B-Instruct				Mistral-7B-Instruct-v0.3				GPT-OSS-20B				Qwen3-4B-Instruct-2507			
		PoisonedRAG		ObliInjection		PoisonedRAG		ObliInjection		PoisonedRAG		ObliInjection		PoisonedRAG		ObliInjection	
		FPR	FNR	FPR	FNR	FPR	FNR	FPR	FNR	FPR	FNR	FPR	FNR	FPR	FNR	FPR	FNR
HotpotQA	PPL	0.36	0.27	0.46	0.47	0.44	0.35	0.52	0.38	0.42	0.61	0.24	0.22	0.47	0.46	0.48	0.37
	KAD	0.58	0.38	0.44	0.43	0.84	0.16	0.79	0.08	0.71	0.33	0.68	0.44	0.72	0.28	0.48	0.37
	EVD	0.78	0.63	0.83	0.60	0.89	0.74	0.78	0.81	0.94	0.00	0.00	0.89	0.63	0.47	0.75	0.13
	DataSentinel	0.03	0.44	0.28	0.02	0.01	0.43	0.28	0.01	0.06	0.39	0.28	0.01	0.05	0.40	0.28	0.01
	ACTPROBE (Ours)	0.00	0.02	0.00	0.02	0.00	0.02	0.00	0.02	0.00	0.06	0.00	0.03	0.00	0.01	0.00	0.03
MS-MARCO	PPL	0.27	0.32	0.42	0.26	0.65	0.45	0.25	0.41	0.45	0.42	0.10	0.15	0.53	0.46	0.36	0.39
	KAD	0.48	0.48	0.55	0.56	0.89	0.08	0.89	0.09	0.74	0.23	0.70	0.54	0.58	0.39	0.76	0.20
	EVD	0.58	0.67	0.55	0.72	0.55	0.82	0.66	0.82	0.73	0.08	0.04	0.73	0.48	0.47	0.44	0.25
	DataSentinel	0.05	0.77	0.06	0.55	0.06	0.78	0.05	0.53	0.09	0.73	0.05	0.58	0.08	0.72	0.05	0.55
	ACTPROBE (Ours)	0.00	0.02	0.00	0.01	0.01	0.02	0.00	0.03	0.02	0.12	0.00	0.05	0.00	0.02	0.00	0.01
Amazon Reviews	PPL	0.35	0.56	0.00	0.00	0.25	0.56	0.52	0.27	0.55	0.48	0.24	0.24	0.62	0.38	0.28	0.66
	KAD	0.98	0.00	0.98	0.00	0.99	0.00	0.99	0.00	0.21	0.78	0.17	0.93	0.79	0.10	0.80	0.19
	EVD	0.84	0.83	0.65	0.74	0.74	0.89	0.56	0.72	0.40	0.74	0.94	0.11	0.45	0.68	0.00	0.79
	DataSentinel	0.00	1.00	0.01	0.96	0.01	0.98	0.00	0.94	0.03	0.97	0.00	0.95	0.03	0.97	0.01	0.96
	ACTPROBE (Ours)	0.04	0.12	0.00	0.02	0.03	0.06	0.00	0.02	0.02	0.14	0.01	0.09	0.05	0.16	0.05	0.14

Table 1: Comprehensive detection performance of different methods against PoisonedRAG and ObliInjection across three datasets and four LLMs. Best results are highlighted in **bold**.

Evaluation Metrics. For detection, following DataSentinel [20], we report False Positive Rate (FPR) and False Negative Rate (FNR), where lower values indicate fewer benign prompts incorrectly flagged and fewer poisoned prompts missed. We additionally report ROC-AUC to evaluate threshold-independent detection performance, where a higher value indicates better discrimination between clean and poisoned prompts. For localization, we report Recall and F1-score to measure whether poisoned segments are correctly recovered while limiting false localization.

5.2 Main Results

5.2.1 Detection Performance

We evaluate whether ACTPROBE can distinguish contaminated prompts from benign ones across three datasets, two attacks, and four LLM architectures. Table 1 reports FPR and FNR, while Figure 7 presents ROC-AUC to assess threshold-independent detection performance. ACTPROBE achieves the best overall balance between false alarms and missed detections, with average FPR and FNR of 0.01 and 0.05, respectively, outperforming PPL (0.38, 0.38), KAD (0.70, 0.29), EVD (0.59, 0.60), and DataSentinel (0.08, 0.61). Its ROC-AUC ranges from 0.95 to 1.00 across all dataset-attack-model combinations, showing that internal activations provide a stable discrimination signal independent of a specific threshold. ACTPROBE remains stable across datasets. On HotpotQA, its FPR is consistently 0.00 and its FNR does not exceed 0.06. It also maintains low errors on MS-MARCO, while keeping FPR and FNR within 0.05 and 0.16, respectively, on the more challenging Amazon Reviews dataset. In contrast, DataSentinel suffers substantial missed detections under domain shifts, with FNRs of 0.53–0.78 on

MS-MARCO and approximately 0.94 on Amazon Reviews. KAD can reduce FNR in some settings but incurs FPRs of approximately 0.98. Overall, ACTPROBE avoids both high false alarms and missed detections, providing more reliable detection across settings.

5.2.2 Localization Performance

We evaluate whether detected attacks can be traced to the responsible input segments. Table 2 reports Recall and F1-score across three datasets, two attacks, and four LLM architectures, where F1-score additionally penalizes over-localization. Across all settings, ACTPROBE achieves an average Recall of 0.94 and F1-score of 0.90, substantially outperforming the strongest baseline, FRA, whose average F1-score is 0.43. The advantage is particularly strong on the QA datasets: ACTPROBE achieves an average F1-score of 0.95 on HotpotQA and average Recall and F1-score of 0.97 and 0.93 on MS-MARCO, respectively. Although Amazon Reviews is more challenging due to its shorter, noisier, and semantically overlapping segments, ACTPROBE obtains the best F1-score in every setting. For example, under ObliInjection on Mistral-7B-Instruct-v0.3, PromptLocate achieves slightly higher Recall than ACTPROBE (0.94 vs. 0.93) but a substantially lower F1-score (0.51 vs. 0.91), indicating over-localization. Overall, ACTPROBE more reliably recovers poisoned segments while limiting false attribution.

5.3 Robustness against Adaptive Attacks

To assess the resilience of ACTPROBE against adaptive adversaries, we evaluate several evasion strategies. Unless otherwise specified, experiments are conducted on MS-MARCO using Llama-3.1-8B-Instruct and PoisonedRAG.

Dataset	Method	Llama-3.1-8B-Instruct				Mistral-7B-Instruct-v0.3				GPT-OSS-20B				Qwen3-4B-Instruct-2507			
		PoisonedRAG		ObliInjection		PoisonedRAG		ObliInjection		PoisonedRAG		ObliInjection		PoisonedRAG		ObliInjection	
		Recall	F1-score	Recall	F1-score	Recall	F1-score	Recall	F1-score	Recall	F1-score	Recall	F1-score	Recall	F1-score	Recall	F1-score
HotpotQA	SFA	0.22	0.19	0.08	0.05	0.10	0.09	0.02	0.01	0.42	0.33	0.29	0.19	0.26	0.23	0.01	0.01
	FRA	0.49	0.36	0.76	0.51	0.46	0.36	0.67	0.45	0.77	0.52	0.66	0.44	0.79	0.52	0.23	0.16
	SVA	0.27	0.22	0.36	0.24	0.07	0.06	0.09	0.06	0.47	0.36	0.32	0.22	0.55	0.42	0.03	0.02
	PromptLocate	0.71	0.45	1.00	0.35	0.69	0.45	1.00	0.36	0.71	0.45	0.99	0.36	0.71	0.44	0.99	0.36
	ACTPROBE (Ours)	0.97	0.97	1.00	0.90	1.00	0.97	1.00	0.98	0.98	0.97	1.00	1.00	0.98	0.93	0.90	0.87
MS-MARCO	SFA	0.28	0.25	0.00	0.00	0.13	0.12	0.06	0.04	0.43	0.35	0.27	0.18	0.23	0.21	0.03	0.02
	FRA	0.64	0.48	0.92	0.61	0.64	0.48	0.92	0.61	0.79	0.56	0.79	0.53	0.87	0.60	0.83	0.55
	SVA	0.33	0.29	0.16	0.11	0.18	0.16	0.25	0.16	0.47	0.38	0.65	0.43	0.52	0.41	0.33	0.22
	PromptLocate	0.42	0.36	0.97	0.40	0.43	0.36	0.98	0.38	0.45	0.37	0.97	0.39	0.45	0.38	0.99	0.41
	ACTPROBE (Ours)	0.98	0.93	1.00	0.94	0.98	0.96	1.00	0.99	0.87	0.85	0.99	0.98	0.98	0.92	0.99	0.87
Amazon Reviews	SFA	0.87	0.55	0.28	0.19	0.74	0.47	0.17	0.11	0.47	0.29	0.17	0.11	0.65	0.40	0.25	0.17
	FRA	0.89	0.56	0.36	0.24	0.88	0.56	0.28	0.19	0.46	0.29	0.24	0.16	0.63	0.37	0.30	0.20
	SVA	0.91	0.57	0.46	0.31	0.86	0.55	0.20	0.14	0.45	0.28	0.26	0.17	0.66	0.41	0.44	0.29
	PromptLocate	0.17	0.09	0.78	0.38	0.25	0.14	0.94	0.51	0.29	0.14	0.91	0.50	0.30	0.14	0.80	0.36
	ACTPROBE (Ours)	0.96	0.94	0.98	0.97	0.98	0.95	0.93	0.91	0.78	0.74	0.72	0.68	0.80	0.71	0.75	0.65

Table 2: Poisoned segment localization performance across three datasets and four LLMs. Recall and F1-score are reported as decimal values. Best results are highlighted in **bold**.

5.3.1 Paraphrase Evasion Attack

Adversarial paraphrasing aims to evade text-level defenses by rewriting poisoned segments while preserving their malicious intent. We evaluate this attack on Llama-3.1-8B-Instruct using MS-MARCO under three paraphrase levels: **L1** replaces key words with semantic equivalents, **L2** restructures sentences without changing their meaning, and **L3** performs semantic reconstruction using an oracle model (*e.g.*, GPT-5.4-mini; see Figure 6) while retaining the adversarial payload.

Table 3 shows that paraphrasing substantially weakens surface-level and external-representation defenses. For detection, PPL’s FNR increases from 0.32 to 0.63 from the original attack to L3, while EVD remains highly vulnerable, rising from 0.67 to 0.85. DataSentinel also exhibits high FNR under paraphrasing, increasing from 0.77 to 0.82. In contrast, ACTPROBE maintains the lowest or tied-lowest FNR across all settings, ranging from 0.02 to 0.18. For localization, PromptLocate drops from 0.36 F1-score on the original attacks to 0.15 under L1 and L2, indicating that paraphrased poisoned segments are difficult to attribute using surface inconsistency. FRA and SVA are more stable but remain below 0.60 F1-score. By comparison, ACTPROBE maintains high localization quality, with F1-score between 0.93 and 0.97. These results indicate that effective paraphrased attacks still leave activation-space traces that can be captured by the poison-direction projection and localized by BINROL.

5.3.2 Signal Dilution and Poisoning Density

We evaluate two adaptive strategies that alter the distribution of malicious content. Signal dilution embeds a short malicious payload into a larger benign passage, with the adversarial

Task Category	Method	Orig	L1	L2	L3
Detection (Metric: FNR ↓)	PPL	0.32	0.50	0.55	0.63
	KAD	0.48	0.44	0.46	0.43
	EVD	0.67	0.80	0.80	0.85
	DataSentinel	0.77	0.81	0.81	0.82
	ACTPROBE (Ours)	0.02	0.13	0.09	0.18
Localization (Metric: F1-score ↑)	SFA	0.25	0.08	0.07	0.07
	FRA	0.48	0.57	0.56	0.58
	SVA	0.29	0.44	0.45	0.47
	PromptLocate	0.36	0.15	0.15	0.19
	ACTPROBE (Ours)	0.94	0.97	0.96	0.93

Table 3: Robustness against paraphrase evasion using Llama-3.1-8B-Instruct on the MS-MARCO dataset.

content ratio decreasing from 100% to 20%. Density-based adaptation increases the number of poisoned segments from $k = 1$ to $k = 5$ with $n = 10$, where $k = 5$ represents a stress test with half of the context contaminated. All experiments use Llama-3.1-8B-Instruct on MS-MARCO.

As shown in Figure 3(a), signal dilution primarily affects detection. At the strongest dilution level, FPR increases from 0.00 to 0.11 and FNR from 0.02 to 0.13, with FNR peaking at 0.18 for the 40% ratio. Localization remains stable, retaining Recall and F1-score of 0.90 at the 20% ratio. In contrast, increasing poisoning density has a greater impact on localization, as shown in Figure 3(b). From $k = 1$ to $k = 5$, FNR increases from 0.02 to 0.15, while Recall and F1-score decrease from 0.98 and 0.93 to 0.62 and 0.73, respectively. This degradation occurs because dense poisoning weakens the minority-outlier assumption of MAD-based leaf detection, allowing multiple poisoned segments within the same region

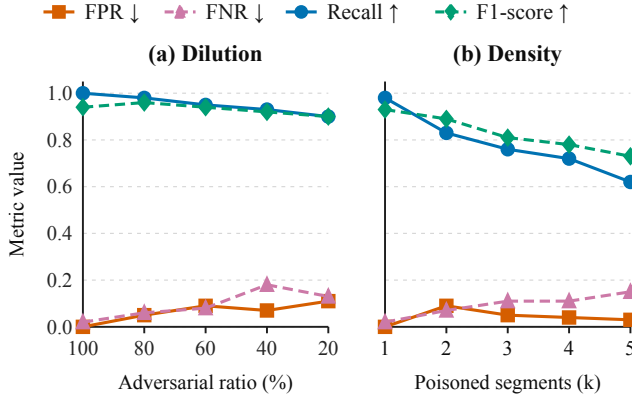


Figure 3: Robustness of ACTPROBE under signal dilution and increasing poisoning density. Lower FPR/FNR and higher Recall/F1-score are better.

to shift the MAD reference and mask weaker signals. Nevertheless, ACTPROBE retains an F1-score of 0.73 even when half of the context is contaminated.

Beyond adaptive paraphrasing, signal dilution, and increased poisoning density, we evaluate a stronger defense-aware attacker that jointly targets the SVM detection score and the BINROL localization signal. With all defense parameters frozen, ACTPROBE retains average detection FPR/FNR of 0.03/0.12 and localization Recall/F1-score of 0.90/0.83. Detailed attack construction and results are provided in Appendix B.2.

5.4 Transferability of ACTPROBE

We evaluate the transferability of ACTPROBE across unseen attacks, data distributions, and LLM architectures. Cross-attack and cross-dataset experiments use Llama-3.1-8B-Instruct to isolate the transfer variable (Sections 5.4.1 and 5.4.2), while cross-architecture adaptation is evaluated separately in Section 5.4.3.

5.4.1 Cross-Attack Transferability

To characterize the transferability of the learned poison direction $\mathbf{v}_p$ across attack families, we evaluate the empirical zero-shot transfer of ACTPROBE between PoisonedRAG (PR, knowledge poisoning) and ObliInjection (OI, prompt injection). Experiments use Llama-3.1-8B-Instruct on MS-MARCO. As shown in Table 4, transfer is asymmetric. When calibrated on PR and evaluated on OI, ACTPROBE achieves a localization F1-score of 0.79, while detection FNR increases to 0.35. In the reverse direction, FNR increases to 0.64 and localization F1-score decreases to 0.66. This asymmetry suggests that PR calibration captures broader activation shifts induced by corrupted evidence, whereas OI calibration emphasizes more attack-specific instruction triggers. Thus, the

Calibration (Train)	Evaluation (Test)	Detection Phase		Localization Phase	
		FPR ($\downarrow$)	FNR ($\downarrow$)	Recall ($\uparrow$)	F1-score ($\uparrow$)
PR	PR	0.00	0.02	0.98	0.93
PR	OI	0.21	0.35	0.79	0.79
OI	OI	0.00	0.01	1.00	0.94
OI	PR	0.00	0.64	0.66	0.66

Table 4: Cross-attack transferability of ACTPROBE on Llama-3.1-8B-Instruct using MS-MARCO. PR denotes PoisonedRAG and OI denotes ObliInjection.

Calibration Data	Test: HotpotQA		Test: MS-MARCO		Test: Amazon	
	FNR	F1-score	FNR	F1-score	FNR	F1-score
HotpotQA	0.02	0.97	0.01	0.96	0.00	0.69
MS-MARCO	0.02	0.95	0.02	0.93	0.00	0.74
Amazon Reviews	0.18	0.70	0.23	0.84	0.12	0.94

Table 5: Cross-dataset transferability of ACTPROBE against PoisonedRAG on Llama-3.1-8B-Instruct.

learned poison direction exhibits partial cross-attack transfer rather than universal attack independence, and performance may degrade against unseen attacks with substantially different activation patterns. Appendix B.3 further evaluates transfer between distinct attack objectives, including answer manipulation and tool-use hijacking.

5.4.2 Cross-Dataset Transferability

To evaluate zero-shot transfer under shifts in contextual semantics and segment length, we calibrate ACTPROBE on one dataset and evaluate it on others using Llama-3.1-8B-Instruct under PoisonedRAG. As shown in Table 5, ACTPROBE transfers well between the two QA datasets: calibration on HotpotQA yields an FNR of 0.01 and localization F1-score of 0.96 on MS-MARCO. Calibration on either QA dataset also transfers effectively to Amazon Reviews for detection, maintaining an FNR of 0.00, although localization performance declines under the domain shift. In contrast, calibration on Amazon Reviews transfers less effectively to QA, with FNR increasing to 0.18 on HotpotQA and 0.23 on MS-MARCO. This asymmetry suggests that contextually rich QA data provide broader activation coverage, whereas the narrower review domain produces a more specialized poison direction. Thus, diverse calibration data can improve transfer across heterogeneous applications.

5.4.3 Cross-Architecture Adaptation (Few-Shot)

Because hidden dimensions differ across LLM architectures, the poison direction $\mathbf{v}_p$ cannot be transferred directly and must be re-estimated for each model. We therefore re-calibrate ACTPROBE on each target architecture using only $M = 50$

Target LLM Architecture	Detection Phase		Localization Phase	
	FPR ($\downarrow$)	FNR ($\downarrow$)	Recall ($\uparrow$)	F1-score ($\uparrow$)
Llama-3.1-8B-Instruct	0.00	0.08	0.98	0.88
Mistral-7B-Instruct-v0.3	0.00	0.07	0.97	0.94
Qwen3-4B-Instruct-2507	0.01	0.06	0.99	0.75
GPT-OSS-20B	0.02	0.14	0.84	0.78

Table 6: Few-shot cross-architecture adaptation of ACTPROBE. The poison direction $\mathbf{v}_p$ is re-calibrated on each target LLM using MS-MARCO samples with PoisonedRAG.

samples and evaluate it on MS-MARCO under PoisonedRAG. As shown in Table 6, few-shot calibration achieves an FNR of 0.07 and F1-score of 0.94 on Mistral-7B-Instruct-v0.3. On GPT-OSS-20B, ACTPROBE retains an F1-score of 0.78, although FNR increases to 0.14. These results show that PoisonedRAG-induced activation shifts remain sufficiently structured for efficient adaptation across open-weight LLMs with limited re-calibration overhead.

5.5 Ablation Studies

We evaluate the contribution of each ACTPROBE component using Llama-3.1-8B-Instruct on MS-MARCO under ObliInjection, unless otherwise specified. In the following tables, HQA, MAR, and AMZ denote HotpotQA, MS-MARCO, and Amazon Reviews, respectively.

5.5.1 Classifier and Decision Logic Comparison

We compare Linear SVM with training-free rules (Best Single Layer, Mean Fusion, and Max Fusion) and supervised classifiers, including Logistic Regression (LR) [9], Random Forest (RF) [4], and RBF-SVM [8]. As shown in Table 7, Linear SVM achieves a tied-best average FNR of 0.02 and the lowest FNR on MS-MARCO (0.01). Training-free rules are less stable, with average FNRs of 0.03–0.05; Max Fusion reaches 0.08 on MS-MARCO. RF also obtains an average FNR of 0.02, but Linear SVM offers comparable performance with a simpler decision boundary. RBF-SVM performs worst, with an average FNR of 0.11, indicating possible overfitting under limited calibration data. These results show that projection onto the poison direction produces a sufficiently structured signal for lightweight linear classification.

5.5.2 Efficacy of the Poison Direction $\mathbf{v}_p$

We assess whether the learned poison direction captures attack-relevant signals by comparing its projection with Random Direction, PCA top- k components, and unreduced MLP activations on Llama-3.1-8B-Instruct using MS-MARCO. As shown in Table 8, $\mathbf{v}_p$ achieves the best performance, with an FNR of 0.01, Recall of 1.00, and F1-score of 0.94. PCA

Category	Method / Classifier	HQA	MAR	AMZ	Avg
Heuristic Rules	Best Single Layer	0.02	0.04	0.04	0.03
	Mean Fusion	0.04	0.06	0.04	0.05
	Max Fusion	0.05	0.08	0.03	0.05
Supervised Classifiers	Logistic Regression (LR)	0.02	0.04	0.02	0.03
	Random Forest (RF)	0.02	0.03	0.02	0.02
	Non-linear SVM (RBF)	0.12	0.12	0.08	0.11
	Linear SVM (Ours)	0.02	0.01	0.02	0.02

Table 7: Detection-phase classifier and decision-logic comparison on Llama-3.1-8B-Instruct. The metric is FNR ($\downarrow$).

Dimensionality Reduction	Detection Phase		Localization Phase	
	FPR ($\downarrow$)	FNR ($\downarrow$)	Recall ($\uparrow$)	F1-score ($\uparrow$)
Random Direction	0.04	0.19	0.44	0.36
PCA (top- k components)	0.00	0.45	0.20	0.14
No Reduction (Raw acts)	0.00	0.02	0.72	0.68
$\mathbf{v}_p$ Projection (Ours)	0.00	0.01	1.00	0.94

Table 8: Impact of dimensionality reduction methods on detection and localization on MS-MARCO.

yields an FNR of 0.45 and F1-score of 0.14, suggesting that variance-dominant components primarily capture benign semantic variation. Random Direction also provides weak localization, with an F1-score of 0.36. Raw activations retain useful information and achieve an F1-score of 0.68, but incur substantially higher computational overhead. These results indicate that the learned direction effectively reduces task-irrelevant variation while preserving poisoning-related activation shifts.

5.5.3 Necessity of Replacement Ablation

During the coarse-grained search, BINROL masks candidate segments to isolate their contribution to the poisoning signal. We compare replacement ablation, which substitutes candidates with neutral text, against deletion ablation using Llama-3.1-8B-Instruct. As shown in Table 9, replacement consistently performs better across datasets, improving average localization F1-score from 0.84 to 0.94 and from 0.83 to 0.94 on HotpotQA. Deletion alters sequence structure and token positions, introducing unrelated activation changes, whereas replacement preserves positional context and enables more reliable attribution.

5.5.4 Robustness of MAD-based Leaf Detection

We compare MAD-based leaf attribution in BINROL with a conventional z -score baseline using Llama-3.1-8B-Instruct on MS-MARCO, under sparse ($k = 1$) and dense ($k = 3$) poisoning with $n = 10$. As shown in Table 10, all variants achieve a Recall of 1.00 for $k = 1$. Under dense poisoning,

Ablation Strategy	HQA	MAR	AMZ	Average
Deletion (Physical Masking)	0.83	0.82	0.88	0.84
Replacement (Ours)	0.94	0.90	0.97	0.94

Table 9: Replacement versus deletion ablation for localization using Llama-3.1-8B-Instruct. The metric is F1-score ($\uparrow$).

Leaf Detection Strategy	Sparse ($k=1$)		Dense ($k=3$)	
	Recall ($\uparrow$)	F1-score ($\uparrow$)	Recall ($\uparrow$)	F1-score ($\uparrow$)
z -score ($z=0.5$)	1.00	1.00	0.40	0.54
z -score ($z=1.0$)	1.00	0.99	0.32	0.46
MAD ($\gamma=0.3$)	1.00	0.67	0.53	0.64
MAD ($\gamma=0.5$)	1.00	0.75	0.60	0.68
MAD ($\gamma=0.80$, Ours)	1.00	0.94	0.62	0.68

Table 10: Localization performance under different outlier detection strategies on MS-MARCO.

however, the z -score baseline with $z = 0.50$ drops to a Recall of 0.40 and an F1-score of 0.54. MAD with $\gamma = 0.80$ achieves the highest Recall of 0.62 and a tied-best F1-score of 0.68 for $k = 3$, while retaining an F1-score of 0.94 for $k = 1$. This robustness stems from median-based statistics. When multiple poisoned candidates occur in the same leaf, their high ablation scores can shift the mean and inflate the standard deviation, raising the z -score threshold and masking poisoned segments. By relying on the median, MAD is less sensitive to extreme scores when poisoned candidates do not dominate the leaf, making it more reliable for multi-segment attribution.

6 Discussion and Limitations

6.1 Surrogate-Based Black-Box Defense

ACTPROBE requires internal MLP activations, which are unavailable for closed-source API models. To support this setting, we deploy an open-weight LLM as an *upstream diagnostic surrogate*. The surrogate detects and localizes poisoned segments, removes them from the prompt, and forwards the sanitized prompt to the target API for final generation, without accessing or modifying the target model’s internal states. End-to-end evaluations against ObliInjection show that this surrogate defense consistently suppresses attacks while preserving task utility. Across all settings, it reduces average ASR from 0.29 to 0.02 and increases ACC from 0.71 to 0.87, whereas budget-matched random removal provides little benefit. Detailed metric definitions, experimental settings, and per-setting results are provided in Appendix B.1.

6.2 System Efficiency and Practicality

Data-efficient and maintainable calibration. ACTPROBE uses a small labeled calibration set to learn the poison di-

rection $\mathbf{v}_p$, train the detector, and estimate the benign distribution parameters (μ, Σ^{-1}) . We use $M = 100$ samples per class, corresponding to 100 benign and 100 poisoned prompts. Benign samples can be collected from operational data, while poisoned samples can be synthesized using known attacks or red-teaming. A sensitivity analysis with $M \in \{10, 25, 50, 100, 200\}$ shows that detection improves substantially up to $M = 100$ and largely saturates thereafter, while localization remains stable across calibration sizes. Detailed results are provided in Appendix B.4. Calibration is performed offline, and representative samples from anticipated attacks and data domains can be pooled to train a single detector for each protected model or application context. The calibration set can be periodically updated as new attacks emerge, while runtime detection overhead remains independent of the number of attack types represented during calibration.

Computational scalability of localization. Table 16 in the Appendix B compares BINROL with four localization baselines under $n = 50$. SFA requires one forward pass but achieves a low F1-score of 0.25. FRA performs segment-wise deletion, requiring 51 forward passes and achieving an F1-score of 0.48. SVA has exponential complexity, while PromptLocate depends on oracle LLM calls and thus has non-comparable overhead. In contrast, BINROL requires 21 forward passes, achieving a $2.43\times$ speedup over FRA and a higher F1-score of 0.93. These results show that BINROL improves both localization efficiency and accuracy, making ACTPROBE practical for multi-source LLM agents.

6.3 Limitations and Future Work

The current ACTPROBE targets segment-level poisoning whose influence remains attributable through segment replacement. Although effective under both sparse and dense poisoning, localization accuracy decreases as the number of poisoned segments grows, motivating improved attribution under higher poisoning densities. Ultra-long contexts may diffuse poisoning signals across layers and token positions, reducing the effectiveness of last-token probing. Future work could explore multi-layer and token-wise probes to capture earlier transient signals and improve long-context robustness.

7 Conclusion

We presented ACTPROBE, an internal-state-based defense for detecting and localizing poisoned segments in multi-source LLM inputs. ACTPROBE leverages a learned poison direction to detect contamination and uses BINROL to efficiently localize poisoned segments through replacement ablation and robust leaf detection. Experiments across datasets, attacks, and LLM architectures show that ACTPROBE outperforms existing detection and localization baselines, remains robust against adaptive evasion, and can protect black-box APIs via an upstream surrogate model.

References

- [1] Hervé Abdi and Lynne J Williams. Principal component analysis. *Wiley interdisciplinary reviews: computational statistics*, 2(4):433–459, 2010.
- [2] Gabriel Alon and Michael Kamfonas. Detecting language model attacks with perplexity. *arXiv preprint arXiv:2308.14132*, 2023.
- [3] Payal Bajaj, Daniel Campos, Nick Craswell, Li Deng, Jianfeng Gao, Xiaodong Liu, Rangan Majumder, Andrew McNamara, Bhaskar Mitra, Tri Nguyen, et al. Ms marco: A human generated machine reading comprehension dataset. 2016.
- [4] Leo Breiman. Random forests. *Machine learning*, 45(1):5–32, 2001.
- [5] Collin Burns, Haotian Ye, Dan Klein, and Jacob Steinhardt. Discovering latent knowledge in language models without supervision. *arXiv preprint arXiv:2212.03827*, 2022.
- [6] Sizhe Chen, Julien Piet, Chawin Sitawarin, and David Wagner. {StruQ}: Defending against prompt injection with structured queries. In *34th USENIX Security Symposium (USENIX Security 25)*, pages 2383–2400, 2025.
- [7] Sizhe Chen, Arman Zharmagambetov, Saeed Mahlouljifar, Kamalika Chaudhuri, David Wagner, and Chuan Guo. Secalign: Defending against prompt injection with preference optimization. In *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security*, pages 2833–2847, 2025.
- [8] Corinna Cortes and Vladimir Vapnik. Support-vector networks. *Machine learning*, 20(3):273–297, 1995.
- [9] David R Cox. The regression analysis of binary sequences. *Journal of the Royal Statistical Society Series B: Statistical Methodology*, 20(2):215–232, 1958.
- [10] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.
- [11] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you’ve signed up for: Compromising real-world llm-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM workshop on artificial intelligence and security*, pages 79–90, 2023.
- [12] Neel Jain, Avi Schwarzschild, Yuxin Wen, Gowthami Somepalli, John Kirchenbauer, Ping-yeh Chiang, Micah Goldblum, Aniruddha Saha, Jonas Geiping, and Tom Goldstein. Baseline defenses for adversarial attacks against aligned language models. *arXiv preprint arXiv:2309.00614*, 2023.
- [13] Yuqi Jia, Yupei Liu, Zedian Shao, Jinyuan Jia, and Neil Gong. Promptlocate: Localizing prompt injection attacks. *arXiv preprint arXiv:2510.12252*, 2025.
- [14] Yuqi Jia, Zhaorun Shao, Yupei Liu, Jinyuan Jia, Dawn Song, and Neil Zhenqiang Gong. A critical evaluation of defenses against prompt injection attacks. *arXiv preprint arXiv:2505.18333*, 2025.
- [15] Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, L  lio Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timoth  e Lacroix, and William El Sayed. Mistral 7b. *arXiv preprint arXiv:2310.06825*, 2023.
- [16] Yang Jiao, Xiaodong Wang, and Kai Yang. Pr-attack: Coordinated prompt-rag attacks on retrieval-augmented generation in large language models via bilevel optimization. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 656–667, 2025.
- [17] John Law. Robust statistics—the approach based on influence functions, 1986.
- [18] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich K  ttler, Mike Lewis, Wen-tau Yih, Tim Rockt  schel, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems*, 33:9459–9474, 2020.
- [19] Yupei Liu, Yuqi Jia, Runpeng Geng, Jinyuan Jia, and Neil Zhenqiang Gong. Formalizing and benchmarking prompt injection attacks and defenses. In *33rd USENIX Security Symposium (USENIX Security 24)*, pages 1831–1847, 2024.
- [20] Yupei Liu, Yuqi Jia, Jinyuan Jia, Dawn Song, and Neil Zhenqiang Gong. Datasentinel: A game-theoretic detection of prompt injection attacks. In *2025 IEEE Symposium on Security and Privacy (SP)*, pages 2190–2208. IEEE, 2025.
- [21] Scott M Lundberg and Su-In Lee. A unified approach to interpreting model predictions. *Advances in neural information processing systems*, 30, 2017.
- [22] Prasanta Chandra Mahalanobis. On the generalized distance in statistics. *Sankhy  : The Indian Journal of Statistics, Series A (2008-)*, 80:S1–S7, 2018.

- [23] Yohei Nakajima. Yohei’s blog post. Twitter, 2022. URL: <https://twitter.com/yoheinakajima/status/1582844144640471040>.
- [24] Dario Pasquini, Martin Strohmeier, and Carmela Troncoso. Neural exec: Learning (and learning from) execution triggers for prompt injection attacks. In *Proceedings of the 2024 Workshop on Artificial Intelligence and Security*, pages 89–100, 2024.
- [25] Vitali Petsiuk, Abir Das, and Kate Saenko. Rise: Randomized input sampling for explanation of black-box models. *arXiv preprint arXiv:1806.07421*, 2018.
- [26] Liz Reid. Generative AI in search: Let Google do the searching for you. Google Blog, 2024. URL: <https://blog.google/products/search/generative-ai-google-search-may-2024/>.
- [27] Vaughn Schermerhorn. How Amazon continues to improve the customer reviews experience with generative AI. About Amazon, 2023. URL: <https://www.aboutamazon.com/news/amazon-ai/amazon-improves-customer-reviews-with-generative-ai>.
- [28] Avital Shafra, Roei Schuster, and Vitaly Shmatikov. Machine against the {RAG}: Jamming {Retrieval-Augmented} generation with blocker documents. In *34th USENIX Security*, pages 3787–3806, 2025.
- [29] Jiawen Shi, Zenghui Yuan, Yinuo Liu, Yue Huang, Pan Zhou, Lichao Sun, and Neil Zhenqiang Gong. Optimization-based prompt injection attack to llm-as-a-judge. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, pages 660–674, 2024.
- [30] Jiawen Shi, Zenghui Yuan, Guiyao Tie, Pan Zhou, Neil Zhenqiang Gong, and Lichao Sun. Prompt injection attack to tool selection in llm agents. *arXiv preprint arXiv:2504.19793*, 2025.
- [31] Xue Tan, Hao Luan, Mingyu Luo, Xiaoyan Sun, Ping Chen, and Jun Dai. RevPRAG: Revealing poisoning attacks in retrieval-augmented generation through LLM activation analysis. *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 12999–13011, 2025.
- [32] Nandan Thakur, Nils Reimers, Andreas Rücklé, Abhishek Srivastava, and Iryna Gurevych. Beir: A heterogeneous benchmark for zero-shot evaluation of information retrieval models. *arXiv preprint arXiv:2104.08663*, 2021.
- [33] Reachal Wang, Yuqi Jia, and Neil Zhenqiang Gong. Obliinjection: Order-oblivious prompt injection attack to llm agents with multi-source data. *arXiv preprint arXiv:2512.09321*, 2025.
- [34] Xilong Wang, John Bloch, Zedian Shao, Yuepeng Hu, Shuyan Zhou, and Neil Zhenqiang Gong. Webinject: Prompt injection attack to web agents. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 2010–2030, 2025.
- [35] Chong Xiang, Tong Wu, Zexuan Zhong, David Wagner, Danqi Chen, and Prateek Mittal. Certifiably robust rag against retrieval corruption. *arXiv preprint arXiv:2405.15556*, 2024.
- [36] Jiaqi Xue, Mengxin Zheng, Yebowen Hu, Fei Liu, Xun Chen, and Qian Lou. Badrag: Identifying vulnerabilities in retrieval augmented generation of large language models. *arXiv preprint arXiv:2406.00083*, 2024.
- [37] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- [38] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William Cohen, Ruslan Salakhutdinov, and Christopher D Manning. Hotpotqa: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of the 2018 conference on empirical methods in natural language processing*, pages 2369–2380, 2018.
- [39] Matthew D Zeiler and Rob Fergus. Visualizing and understanding convolutional networks. In *European conference on computer vision*, pages 818–833. Springer, 2014.
- [40] Baolei Zhang, Yuxi Chen, Zhuqing Liu, Lihai Nie, Tong Li, Zheli Liu, and Minghong Fang. Practical poisoning attacks against retrieval-augmented generation. In *Proceedings of the 31st ACM Symposium on Access Control Models and Technologies*, pages 33–44, 2026.
- [41] Collin Zhang, Tingwei Zhang, and Vitaly Shmatikov. Adversarial decoding: Generating readable documents for adversarial objectives. In *Findings of the Association for Computational Linguistics: EACL 2026*, pages 2053–2068, 2026.
- [42] Andy Zou, Long Phan, Sarah Chen, James Campbell, Phillip Guo, Richard Ren, Alexander Pan, Xu Wang Yin, Mantas Mazeika, Ann-Kathrin Dombrowski, et al. Representation engineering: A top-down approach to ai transparency. *arXiv preprint arXiv:2310.01405*, 2023.
- [43] Wei Zou, Runpeng Geng, Binghui Wang, and Jinyuan Jia. {PoisonedRAG}: Knowledge corruption attacks to {Retrieval-Augmented} generation of large language models. In *34th USENIX Security Symposium (USENIX Security 25)*, pages 3827–3844, 2025.

Ethical Considerations

We structure our ethical analysis around the affected stakeholders, the potential benefits and risks of the research, the safeguards adopted during evaluation, and the justification for publication.

Stakeholders and Research Process. Relevant stakeholders include users of LLM-based applications, model and service providers, contributors to external data sources, and the research community. Segment-level poisoning may expose users to misinformation, manipulated responses, or unintended tool use, while imposing additional security and operational costs on providers. Our experiments use public benchmark datasets, established attack methods, and synthetically constructed attack variants. We recruited no human participants, collected no new personal information, and made no attempt to identify or re-identify dataset contributors. Open-weight models were evaluated locally. Black-box APIs were queried only through standard inference interfaces using benchmark inputs, without modifying persistent service state. Tool-use hijacking was evaluated through sandboxed mock execution, and no real external actions were performed.

Impact of the Research. The primary benefit of ACTPROBE is improved protection against segment-level poisoning. In addition to detecting contaminated prompts, it localizes malicious segments, enabling targeted sanitization while preserving benign context. Its surrogate-based deployment further extends this protection to closed-source LLM APIs. However, the research also presents dual-use risks: descriptions of defense-aware adaptation and tool-use hijacking may help adversaries refine evasion attempts. False positives or incorrect localization may remove useful information or unfairly implicate benign sources. Moreover, internal activations may encode sensitive properties of deployment inputs if retained without appropriate safeguards.

Mitigations. We restrict all attacks to controlled benchmark environments and do not poison production agents, knowledge bases, or user content. The tool-hijacking evaluation contains no credentials, live-service automation, or externally consequential actions. Experimental results are reported only in aggregate. For practical deployment, we recommend access controls for internal activations, minimal activation retention, periodic calibration updates, and human review before taking consequential actions against users or data sources. Localization results should be treated as evidence of suspicious influence rather than proof that a source contributor acted maliciously.

Ethical Principles and Publication Justification. We considered beneficence, respect for persons, justice, and respect for law and the public interest. The work aims to reduce manipulation and misinformation without exposing participants or interfering with live systems. Its targeted sanitization design also limits unnecessary removal of benign content. Although adaptive evaluations provide incremental offensive

knowledge, they are necessary to characterize the defense honestly and rely on established attack paradigms rather than introducing a new capability against otherwise secure systems. We therefore conclude that the defensive and reproducibility benefits of conducting and publishing this research outweigh the residual risks under the safeguards described above.

Open Science

To facilitate independent validation, we provide the implementation of ACTPROBE, together with preprocessing utilities, experimental configurations, and evaluation scripts. These artifacts support reproduction of our main detection and localization experiments and are available at <https://anonymous.4open.science/r/Actprob-5573/>.

A Additional Details of Experiment

A.1 Neutral Text Templates for Replacement Ablation

As detailed in Section 4.4, our BINROL algorithm employs replacement ablation to mitigate signal leakage and precisely isolate the contribution of individual segments to the adversarial activation shift. To ensure our localization mechanism is robust and not biased toward any specific phrasing, we utilized a diverse pool of semantically neutral strings. During the ablation process, the target segment is uniformly replaced by one of the following neutral templates:

- “No relevant information is available for this query.”
- “This document does not contain useful information.”
- “The content of this source is not applicable.”

Our empirical evaluations demonstrate that the exact choice of neutral text from this pool does not significantly impact the drop ratio (ρ), corroborating the structural robustness of the poison direction probe ($\mathbf{v}_p$).

Symbol	Description
f	Target LLM
p, s^t	Full assembled prompt, target instruction
x_i	Retrieved segment from the i -th source
n, k	Total segments, number of poisoned segments
L, d	Number of transformer layers, MLP hidden dimension
$\mathbf{h}_l \in \mathbb{R}^d$	Last-token MLP activation at layer l
$\mathbf{v}_p[l] \in \mathbb{R}^d$	Poison direction vector at layer l (unit norm)
$\mathbf{s} \in \mathbb{R}^L$	Projection feature: $\mathbf{s} = [\mathbf{h}_l \cdot \mathbf{v}_p[l]]_{l=1}^L$
$\mathbf{w} \in \mathbb{R}^L$	Linear SVM weight vector
μ, Σ	Benign distribution mean and covariance
ρ, a_i	Drop ratio, fine-grained ablation score
δ, γ, ℓ	Drop threshold, MAD coefficient, leaf size

Table 11: Summary of Notations

Dataset	Task	#Segs /Input	Avg. Seg. Len.
HotpotQA [38]	Multi-hop QA	6	~140
MS-MARCO [3]	RAG QA	10	~130
Amazon Reviews [27]	Review Summ.	50	~40

Table 12: Dataset statistics used in our evaluation.

You are a precise question-answering assistant. Base your answer strictly and solely on the provided documents. Do not use any external knowledge. Answer the query briefly and clearly. If the documents do not contain sufficient information to answer the query, respond with exactly "I don't know."

Query: [...]
Documents: [...]

Figure 4: The prompt template for the RAG-based question-answering task.

You are an AI assistant tasked with objective product review analysis. Synthesize the provided customer reviews into a concise summary. Your response must systematically address the overall customer consensus, the main strengths and weaknesses of the product.

Customer Reviews: [...]

Figure 5: The prompt template for the e-commerce review summarization task.

You are an expert text rewriter. Below is an original document and its corresponding query. Your task is to comprehensively paraphrase the document to change its expression and syntactic structure. The rewritten passage must retain the exact core meaning and remain highly relevant to answering the provided query.

Query: [...]
Original document: [...]

Figure 6: The prompt template used for generating paraphrased adversarial samples.

Algorithm 1 The ACTPROBE Operational Pipeline

Input: LLM f , calibration set with M clean and M poisoned prompts, assembled prompt $p = s^d \|x_1\| \dots \|x_n$, number of selected layers κ , drop threshold δ , MAD scaling constant η , MAD coefficient γ , leaf size ℓ , neutral text z

Output: Detection label $\hat{y}$, localized poisoned segments $\mathcal{D}$

```

1: // Phase I: Offline Calibration {Executes once prior to deployment}
2: for each calibration prompt  $p_i$  do
3:   Forward pass through  $f$ ; extract last-token MLP activation  $\mathbf{h}_{i,l}$  at each layer  $l$ 
4: end for
5: for  $l = 1, \dots, L$  do
6:   Compute poison direction:  $\mathbf{v}_l \leftarrow \frac{1}{M} \sum_{i=1}^M \mathbf{h}_{i,l}^p - \frac{1}{M} \sum_{i=1}^M \mathbf{h}_{i,l}^c$ 
7:   Normalize to unit vector:  $\mathbf{v}_p[l] \leftarrow \mathbf{v}_l / \|\mathbf{v}_l\|$ 
8: end for
9: for each calibration prompt  $p_i$  do
10:  Compute projection features:  $\mathbf{s}_i \leftarrow [\mathbf{h}_{i,l} \cdot \mathbf{v}_p[l]]_{l=1}^L$ 
11: end for
12: Fit scaler on  $\{\mathbf{s}_i\}$ ; train linear SVM to obtain weight vector  $\mathbf{w}$ 
13: Estimate benign distribution from unstandardized clean features:
     $\mu \leftarrow \text{mean}(\{\mathbf{s}_i^c\}); \Sigma^{-1} \leftarrow (\text{cov}(\{\mathbf{s}_i^c\}) + \lambda \mathbf{I})^{-1}$ 
14: // Phase II: Online Inference {Executes for each incoming prompt}
15: Forward pass on  $p$ ; extract last-token MLP activations  $\mathbf{h}_l$ 
16: Compute projection features:  $\mathbf{s} \leftarrow [\mathbf{h}_l \cdot \mathbf{v}_p[l]]_{l=1}^L$ 
17:  $\hat{y} \leftarrow \text{SVM}(\text{scaler}(\mathbf{s}))$ 
18: if  $\hat{y} == \text{Benign}$  then
19:   return Benign,  $\emptyset$ 
20: end if
21:  $\mathcal{D} \leftarrow \text{BINROL}(f, p, \{x_1, \dots, x_n\}, \mathbf{v}_p, \mu, \Sigma^{-1}, \mathbf{w}, \kappa, \delta, \eta, \gamma, \ell, z)$ 
    {Alg. 2}
22: return Poisoned,  $\mathcal{D}$ 

```

B Additional Experimental Results

B.1 End-to-End Black-Box Surrogate Defense

To evaluate ACTPROBE on closed-source LLM APIs, an open-weight surrogate detects and removes poisoned segments before forwarding the sanitized prompt to the target API. Table 13 reports ASR and ACC. ASR measures whether the output contains or implies the attacker-specified target fact, manifested as induced answers on HotpotQA and MS-MARCO and injected malicious opinions on Amazon Reviews. ACC measures semantic correctness against reference answers for the two QA datasets and overall summary faithfulness, coverage, and factual correctness for Amazon Reviews. ASR and ACC are evaluated independently rather than as complementary outcomes; an Amazon Reviews summary may retain the main benign content while also including the injected opinion, so their sum can exceed 1. Random Removal uses the same deletion budget as ACTPROBE to isolate the benefit of accurate localization. As shown in Table 13, No Defense and Random Removal both yield an average ASR

of 0.29, with ACC values of 0.71 and 0.70, respectively. In contrast, ACTPROBE reduces the average ASR to 0.02 while increasing ACC to 0.87, with ASR remaining at or below 0.05 across all surrogate–target–dataset combinations. These gains demonstrate that accurately localizing and removing poisoned segments, rather than arbitrary deletion, enables effective upstream protection for closed-source LLM APIs.

Algorithm 2 BINROL: Binary Search with Robust Leaf Detection

Input: LLM f , full assembled prompt p , segments $\{x_i\}_{i=1}^n$, poison direction $\mathbf{v}_p$, benign distribution (μ, Σ^{-1}) , SVM weights $\mathbf{w}$, number of selected layers κ , drop threshold δ , MAD scaling constant η , MAD coefficient γ , leaf size ℓ , neutral text z

Output: Set of detected poison positions $\mathcal{D}$

```

1: // Initialization
2:  $\mathcal{L}_{\text{top}} \leftarrow \text{top-}\kappa \text{ indices of } |\mathbf{w}|$ ;  $\mathcal{D} \leftarrow \emptyset$ 
3:  $C_0 \leftarrow \{1, \dots, n\}$ 
4: Forward pass on  $p$ ; extract last-token MLP activations from all
    $L$  layers and compute  $\mathbf{s}_{\text{full}} = [\mathbf{h}_l^{\text{full}} \cdot \mathbf{v}_p[l]]_{l=1}^L \in \mathbb{R}^L$ 
5:  $d_{\text{full}} \leftarrow d(\mathbf{s}_{\text{full}})$  {Eq. 8}
6: BINARYNARROW( $C_0, C_0, \mathcal{D}$ )
7: return  $\mathcal{D}$ 

8: // Procedure 1: Coarse-grained binary narrowing
9: procedure BINARYNARROW( $C, \mathcal{P}, \mathcal{D}$ )
10: if  $|C| \leq \ell$  then
11:   ROBUSTLEAF( $C, \mathcal{P}, \mathcal{D}$ ); return
12: end if
13: Split  $C$  evenly into halves  $C_L$  and  $C_R$ 
14: for each half  $\mathcal{H} \in \{C_L, C_R\}$  do
15:   Replace segments  $\{x_i\}_{i \in \mathcal{H}}$  with  $z$  in  $p$ 
16:   Compute drop ratio  $\rho_{\mathcal{H}}$  using all-layer projection features.
   {Eq. 9}
17: end for
18:  $\mathcal{H}^* \leftarrow \{\mathcal{H} \in \{C_L, C_R\} \mid \rho_{\mathcal{H}} > \delta\}$ 
19: if  $\mathcal{H}^* \neq \emptyset$  then
20:   for each  $\mathcal{H} \in \mathcal{H}^*$  do BINARYNARROW( $\mathcal{H}, C, \mathcal{D}$ )
21: else
22:    $\mathcal{H}_{\text{max}} \leftarrow \arg \max_{\mathcal{H} \in \{C_L, C_R\}} \rho_{\mathcal{H}}$ 
23:   BINARYNARROW( $\mathcal{H}_{\text{max}}, C, \mathcal{D}$ ) {Fallback branch}
24: end if

25: // Procedure 2: Fine-grained robust leaf detection
26: procedure ROBUSTLEAF( $C, \mathcal{P}, \mathcal{D}$ )
27: for each  $i \in \mathcal{P}$  do
28:   Replace  $x_i$  with  $z$  in  $p$ ; compute ablation score  $a_i$  using layers
    $\mathcal{L}_{\text{top}}$  {Eq. 10}
29: end for
30:  $\tilde{m} \leftarrow \text{median}(\{a_i\}_{i \in \mathcal{P}})$ ;  $\hat{\sigma} \leftarrow \eta \cdot \text{median}(\{|a_i - \tilde{m}|\}_{i \in \mathcal{P}})$ 
31:  $\mathcal{D}_{\text{leaf}} \leftarrow \{i \in C \mid a_i > \tilde{m} + \gamma \cdot \hat{\sigma}\}$  {Apply parent-level MAD to
   leaf candidates}
32: if  $\mathcal{D}_{\text{leaf}} = \emptyset$  then  $\mathcal{D}_{\text{leaf}} \leftarrow \{\arg \max_{i \in C} a_i\}$  {Fallback}
33:  $\mathcal{D} \leftarrow \mathcal{D} \cup \mathcal{D}_{\text{leaf}}$ 

```

B.2 Robustness against Defense-Aware Adaptive Attacks

We evaluate ACTPROBE against a defense-aware attacker with knowledge of both its detection and localization mechanisms. For each poisoned input, the attacker constructs multiple variants with the same malicious objective and selects among successful variants those that jointly reduce the SVM detection score and the replacement-ablation signal used by BINROL. Experiments cover three datasets, two attacks, and two representative LLMs. The poison direction, detector, thresholds, and all other defense parameters remain frozen throughout evaluation. As shown in Table 14, adaptation increases the average FPR/FNR from 0.01/0.03 to 0.03/0.12 and decreases localization Recall/F1-score from 0.98/0.95 to 0.90/0.83. The largest degradation occurs for PR on HotpotQA, where FNR rises from 0.02/0.02 to 0.21/0.25 and Recall falls from 0.97/1.00 to 0.88/0.80 on the two LLMs. In contrast, adaptive OI maintains zero FPR, FNRs of 0.03–0.10, and F1-scores of 0.76–0.94. Thus, defense-aware adaptation weakens ACTPROBE, particularly under PR, but does not eliminate its detection and localization capabilities.

B.3 Cross-Goal Transferability

We examine whether a poison direction learned for one attack goal transfers to another. For both PoisonedRAG (PR) and ObliInjection (OI), we consider two goals: *answer manipulation*, which steers the model toward an attacker-specified answer, and *tool-use hijacking*, which induces the selection of an attacker-specified tool. Experiments are conducted on Llama-3.1-8B-Instruct using MS-MARCO. For each attack mechanism, ACTPROBE is calibrated on one goal and evaluated on either the same or the other goal. To instantiate tool-use hijacking, each clean prompt contains the top-10 tools retrieved from a benign tool repository. Following the method introduced in [30], we add an optimized malicious tool description to the repository and rerun retrieval. We retain only instances in which the malicious tool enters the top-10 results and successfully induces the attacker-specified tool selection without defense.

As shown in Table 15, ACTPROBE performs strongly under same-goal calibration. For answer manipulation, it achieves FNRs of 0.02 and 0.01 and localization F1-scores of 0.93 and 0.94 under PR and OI, respectively. For tool-use hijacking, it obtains an FNR of 0.05 and an F1-score of 0.96 under both attacks. In contrast, cross-goal transfer is limited. Calibrating on tool-use hijacking and evaluating answer manipulation increases the FNR to 1.00, while the F1-score drops to 0.12 for PR and 0.01 for OI. In the reverse direction, the FNR reaches 0.73 for PR and 0.99 for OI, with F1-scores of 0.05 and 0.28, respectively. The FPR remains at or below 0.02 across all settings, indicating that the degradation is primarily caused by missed cross-goal attacks rather than increased false

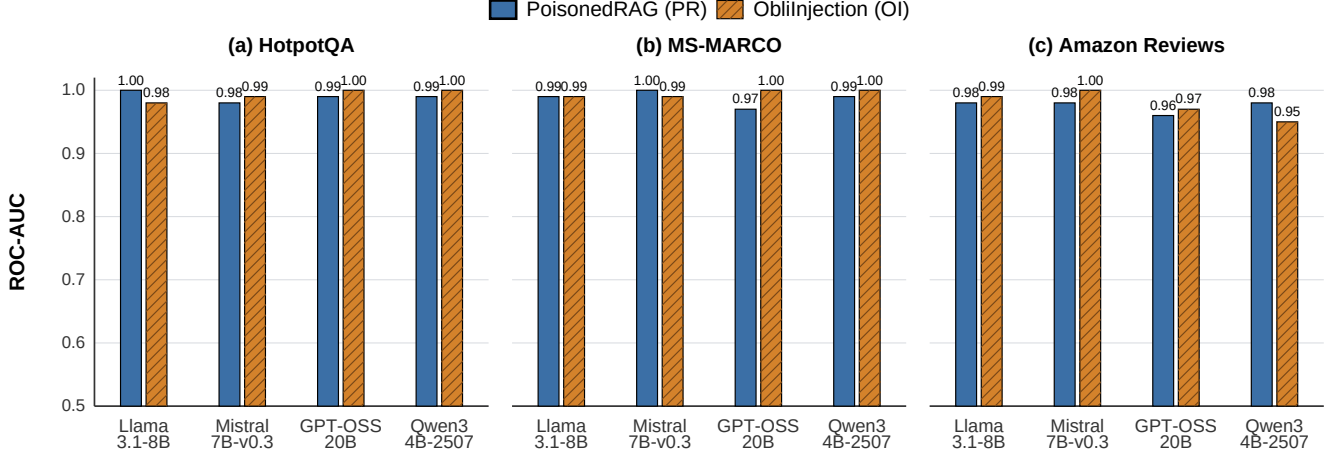


Figure 7: ROC-AUC of ACTPROBE across three datasets, two attacks, and four LLMs. PR denotes PoisonedRAG, and OI denotes ObliInjection. Higher values indicate better detection performance.

Dataset	Surrogate Model	GPT-5.4-mini						Gemini-3.1-flash-Lite					
		No Defense		Random Removal		ACTPROBE		No Defense		Random Removal		ACTPROBE	
		ASR ↓	ACC ↑	ASR ↓	ACC ↑	ASR ↓	ACC ↑	ASR ↓	ACC ↑	ASR ↓	ACC ↑	ASR ↓	ACC ↑
HotpotQA	Llama-3.1-8B-Instruct	0.22	0.68	0.18	0.70	0.00	0.78	0.19	0.71	0.21	0.74	0.01	0.80
	Mistral-7B-Instruct-v0.3			0.21	0.68	0.02	0.82			0.18	0.75	0.00	0.82
	GPT-OSS-20B			0.18	0.71	0.01	0.83			0.23	0.72	0.00	0.86
	Qwen3-4B-Instruct-2507			0.20	0.67	0.00	0.85			0.20	0.69	0.02	0.80
MS-MARCO	Llama-3.1-8B-Instruct	0.28	0.64	0.27	0.66	0.01	0.88	0.24	0.70	0.25	0.71	0.00	0.84
	Mistral-7B-Instruct-v0.3			0.24	0.67	0.04	0.82			0.20	0.72	0.02	0.81
	GPT-OSS-20B			0.20	0.68	0.03	0.79			0.28	0.67	0.04	0.80
	Qwen3-4B-Instruct-2507			0.28	0.62	0.03	0.84			0.22	0.68	0.02	0.81
Amazon Reviews	Llama-3.1-8B-Instruct	0.37	0.80	0.39	0.83	0.00	1.00	0.46	0.70	0.41	0.72	0.00	1.00
	Mistral-7B-Instruct-v0.3			0.42	0.84	0.02	0.98			0.42	0.69	0.03	0.96
	GPT-OSS-20B			0.41	0.72	0.01	0.93			0.43	0.62	0.02	0.95
	Qwen3-4B-Instruct-2507			0.43	0.78	0.03	0.93			0.47	0.63	0.05	0.94

Table 13: End-to-end black-box surrogate defense performance against ObliInjection across datasets, surrogate models, and target APIs. Attack Success Rate (ASR ↓) and task Accuracy (ACC ↑) are reported.

alarms. These results show that the learned poison direction captures goal-conditioned activation shifts; therefore, deployments covering heterogeneous attack goals should include representative calibration samples for each goal or maintain separate goal-specific poison directions.

B.4 Effect of Calibration Set Size

We examine the sensitivity of ACTPROBE to the calibration-set size by varying the number of samples per class as $M \in \{10, 25, 50, 100, 200\}$, where each setting contains M clean and M poisoned prompts. All other experimental settings are kept fixed. Following the main evaluation, detection

and localization are evaluated separately. As shown in Table 17, increasing M substantially improves detection. For PR, the FNR decreases from 0.21 at $M = 10$ to 0.02 at $M = 100$, while for OI it decreases from 0.25 to 0.01. Further increasing M to 200 provides little additional improvement, indicating that detection performance approaches saturation at $M = 100$. Meanwhile, localization remains stable across calibration sizes: PR achieves Recall between 0.95 and 0.98 and F1-score between 0.89 and 0.94, while OI maintains a Recall of 1.00 and F1-score between 0.94 and 0.96. These results support $M = 100$ as a practical balance between calibration cost, detection accuracy, and localization stability.

Dataset	Attack	Attack Version	Llama-3.1-8B-Instruct				Mistral-7B-Instruct-v0.3			
			Detection		Localization		Detection		Localization	
			FPR ($\downarrow$)	FNR ($\downarrow$)	Recall ($\uparrow$)	F1-score ($\uparrow$)	FPR ($\downarrow$)	FNR ($\downarrow$)	Recall ($\uparrow$)	F1-score ($\uparrow$)
HotpotQA	PR	Original	0.00	0.02	0.97	0.97	0.00	0.02	1.00	0.97
		Adaptive	0.07	0.21	0.88	0.84	0.02	0.25	0.80	0.72
	OI	Original	0.00	0.02	1.00	0.90	0.00	0.02	1.00	0.98
		Adaptive	0.00	0.05	0.96	0.89	0.00	0.07	0.92	0.84
MS-MARCO	PR	Original	0.00	0.02	0.98	0.93	0.01	0.02	0.98	0.96
		Adaptive	0.08	0.06	0.97	0.94	0.09	0.16	0.88	0.83
	OI	Original	0.00	0.01	1.00	0.94	0.00	0.03	1.00	0.99
		Adaptive	0.00	0.03	0.96	0.92	0.00	0.08	0.99	0.80
Amazon Reviews	PR	Original	0.04	0.12	0.96	0.94	0.03	0.06	0.98	0.95
		Adaptive	0.02	0.18	0.87	0.81	0.05	0.20	0.75	0.71
	OI	Original	0.00	0.02	0.98	0.97	0.00	0.02	0.93	0.91
		Adaptive	0.00	0.07	0.97	0.94	0.00	0.10	0.81	0.76

Table 14: Detection and localization performance of ACTPROBE against original and adaptive PR and OI attacks. PR denotes PoisonedRAG and OI denotes ObliInjection.

Attack	Cross-Goal Pair			Detection Phase		Localization Phase		
	Evaluation Goal	Calibration Goal	Transfer Type	FPR ($\downarrow$)	FNR ($\downarrow$)	Recall ($\uparrow$)	F1-score ($\uparrow$)	
PR	Answer Manipulation	Answer Manipulation	Same-goal	0.00	0.02	0.98	0.93	
		Tool-Use Hijacking	Cross-goal	0.00	1.00	0.15	0.12	
	Tool-Use Hijacking	Answer Manipulation	Cross-goal	0.02	0.73	0.06	0.05	
		Tool-Use Hijacking	Same-goal	0.00	0.05	0.91	0.96	
OI	Answer Manipulation	Answer Manipulation	Same-goal	0.00	0.01	1.00	0.94	
		Tool-Use Hijacking	Cross-goal	0.00	1.00	0.01	0.01	
	Tool-Use Hijacking	Answer Manipulation	Cross-goal	0.02	0.99	0.44	0.28	
		Tool-Use Hijacking	Same-goal	0.00	0.05	0.91	0.96	

Table 15: Cross-goal transferability of ACTPROBE on Llama-3.1-8B-Instruct using MS-MARCO under PoisonedRAG and ObliInjection. PR and OI denote PoisonedRAG and ObliInjection, respectively.

Localization Strategy	Complexity	Total FP / Calls	Speedup	F1-score
SFA	$O(1)$	1 FP	51.00 $\times$	0.25
FRA	$O(n)$	51 FPs	1.00 $\times$	0.48
SVA	$O(2^n)$ Exact	Exponential	$<1.00\times$	0.29
PromptLocate	$O(k \log n)$ Oracle	Oracle-dependent	Variable	0.36
BINRoL (Ours)	$O(k \log n)$	21 FPs	2.43 $\times$	0.93

Table 16: Efficiency and Performance Comparison of Localization Strategies ($n=50, k=5$). FP denotes Forward Passes. Results are evaluated on Llama-3.1-8B-Instruct and Amazon Reviews.

Attack	Calibration Size M	Detection Phase		Localization Phase	
		FPR ($\downarrow$)	FNR ($\downarrow$)	Recall ($\uparrow$)	F1-score ($\uparrow$)
PR	10	0.00	0.21	0.96	0.94
	25	0.01	0.11	0.95	0.92
	50	0.00	0.06	0.95	0.89
	100	0.00	0.02	0.98	0.93
	200	0.00	0.01	0.98	0.92
OI	10	0.00	0.25	1.00	0.96
	25	0.00	0.13	1.00	0.95
	50	0.00	0.09	1.00	0.96
	100	0.00	0.01	1.00	0.94
	200	0.00	0.01	1.00	0.95

Table 17: Effect of calibration-set size on ACTPROBE using Llama-3.1-8B-Instruct and MS-MARCO. PR denotes PoisonedRAG and OI denotes ObliInjection.

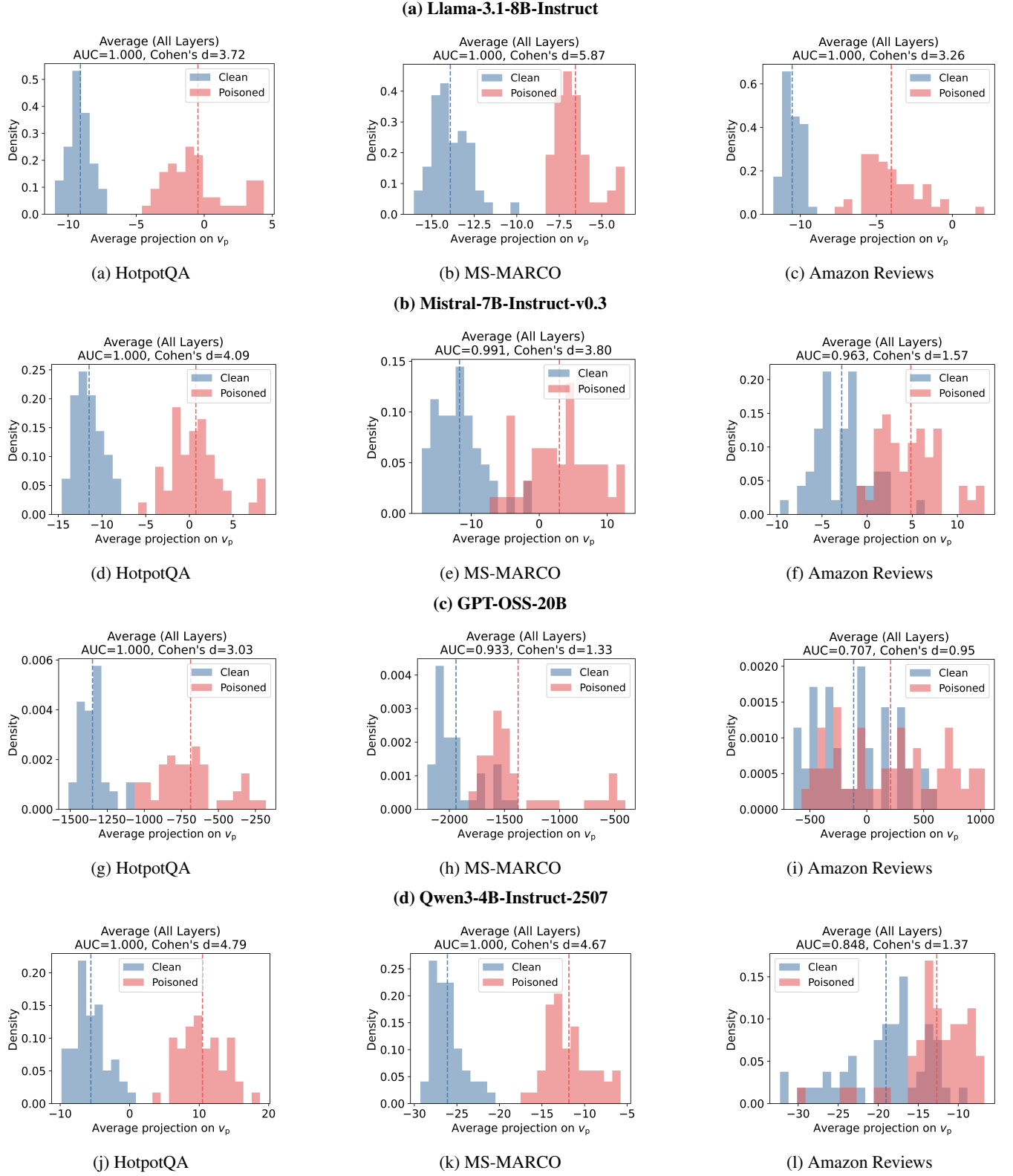


Figure 8: Projection distributions of clean and poisoned activations along the poison direction. Results are shown for four different LLM backbones across three datasets under the ObliInjection attack.

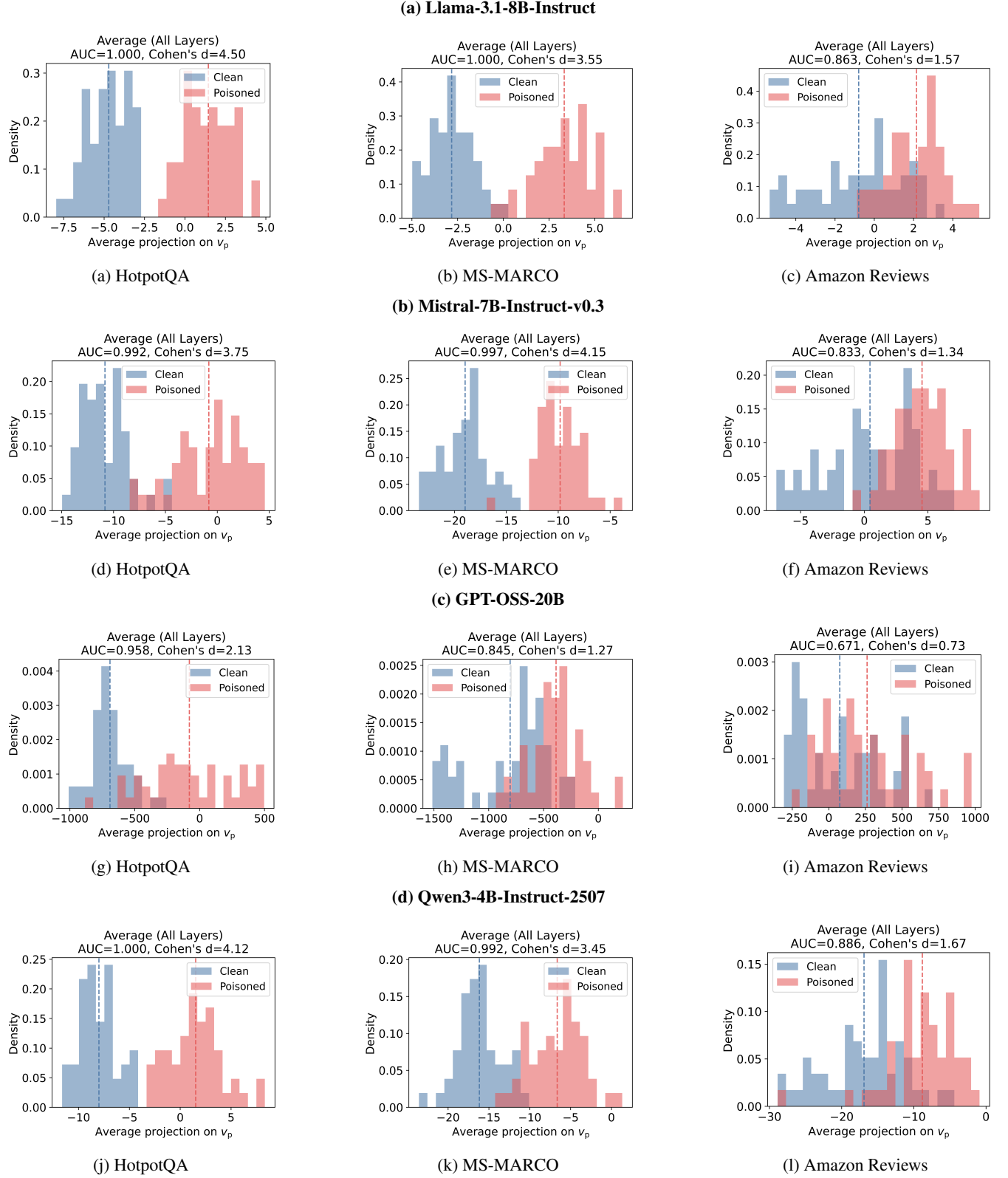


Figure 9: Projection distributions of clean and poisoned activations along the poison direction. Results are shown for four different LLM backbones across three datasets under the PoisonedRAG attack.